

KONERU LAKSHMAIAH EDUCATION FOUNDATION

(Deemed to be University)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

BACHELOR OF TECHNOLOGY PROJECT REPORT

AI-Enhanced Personalized Job Recommendation Platform

CareerIntel – A Hybrid Language-Model and Knowledge-Graph Framework for Skill-Aware Job Recommendation and Labour-Market Analytics

Submitted in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

in Computer Science and Engineering

Student	Register number
N. Harsha Vardhan	2200030767
J. Rakesh	2200031132

Under the supervision of

Dr. K. Gurnadha Gupta

Department of Computer Science and Engineering

KLEF, Green Fields, Vaddeswaram, Andhra Pradesh

Academic Year 2025–26

Declaration

We, N. Harsha Vardhan (2200030767) and J. Rakesh (2200031132), declare that this project report entitled “AI-Enhanced Personalized Job Recommendation Platform” is submitted in partial fulfillment of the requirements for the award of the Bachelor of Technology degree in Computer Science and Engineering at KLEF during the academic year 2025–26.

We declare that the report represents our work and that sources and limitations relevant to the implementation and evaluation are identified. The research extension described as CareerIntel is distinguished from the original capstone implementation wherever appropriate.

N. Harsha Vardhan _____ J. Rakesh _____

Date: _____ Place: KLEF

Certificate

This is to certify that the project report entitled “AI-Enhanced Personalized Job Recommendation Platform” submitted by N. Harsha Vardhan (2200030767) and J. Rakesh (2200031132), in partial fulfillment of the requirements for the award of the Bachelor of Technology degree in Computer Science and Engineering at KLEF during the academic year 2025–26, is a record of project work carried out under my supervision.

Supervisor: Dr. K. Gurnadha Gupta
Signature: _____ Date: _____

Head of the Department: _____ Signature: _____

External Examiner: _____ Signature: _____

Acknowledgements

We express our sincere gratitude to our supervisor, Dr. K. Gurnadha Gupta, for her guidance and academic support during this project. We thank the Department of Computer Science and Engineering and KLEF for providing the academic environment in which the work was undertaken. We also acknowledge the open-source software communities whose libraries are named in the project configuration and dependencies. Any institutional acknowledgements beyond these are left for the authors to complete in accordance with university requirements.

Abstract

This report presents the completed B.Tech project “AI-Enhanced Personalized Job Recommendation Platform” and its research extension, CareerIntel. The GitHub repository contains a Streamlit app, a React/Vite frontend with a FastAPI backend, and ranking code for lexical, semantic, direct-hybrid, and graph-hybrid experiments. The two interfaces call the app-facing recommend_jobs.py module, which uses TF-IDF and weighted skill, requirement, role, and experience signals. The SentenceTransformer and graph-hybrid rankers are separate research components. The processed job and O*NET data needed for a complete run are not included in the repository.

The direct hybrid ranker combines semantic similarity, skill overlap, and title relevance with class defaults of 0.60, 0.30, and 0.10. Earlier saved comparisons cover three profiles and 178 candidates; a separate AI-assisted review comparison covers 180 candidates. The repository also contains graph-hybrid comparisons over three profiles and 135 candidate rows. In the graph-v3 output, the direct hybrid reaches NDCG@10 of 0.8702 and the graph hybrid reaches 0.8699; both have Precision@10 of 0.8333, Recall@10 of 0.5264, and MRR of 1.0000. All graph-v3 labels are marked provisional_assistant. These figures are exploratory, not independently validated ground truth. The job and O*NET inputs are not included, so the results cannot be reproduced from this repository alone. The source includes working interface code, but no user study or production deployment evidence.

Keywords: information retrieval, job recommendation, sentence embeddings, skill normalization, hybrid ranking, knowledge graph, O*NET, ranking evaluation.

Table of Contents

Declaration

Certificate

Acknowledgements

Abstract

List of Figures

List of Tables

Chapter 1 – Introduction

Chapter 2 – Literature Review

Chapter 3 – Problem Definition and System Requirements

Chapter 4 – System Design and Methodology

Chapter 5 – Implementation

Chapter 6 – Results, Evaluation and Discussion

Chapter 7 – Conclusion and Future Work

References

Appendices A–B Repository evidence and metric definitions

Appendices C–I Data dictionary, experiment records, risk and system profile

Appendices J–Q Evaluation protocols, component notes and research roadmap

Appendices R–Z Data governance, field handling, system documentation and reproducibility

Appendices AA–AQ Test cases, technical analysis and evidence-based conclusions

List of Figures

Figure 4.1 Source-level application and recommendation workflow (*processed data not included)

Figure 4.2 Evaluation pipeline used in saved ranking comparisons

Figure 4.3 Knowledge-graph entity and relation schema

Figures H.7–H.15 System and data diagrams, source-tree view, skill and market visualizations, and profile screenshot retained from the original capstone report

Figures H.1–H.6 Dashboard, skill and market-analysis screenshots retained from the original capstone report

List of Tables

Table 3.1 Functional requirements

Table 3.2 Non-functional requirements

Table 5.1 Implemented modules and application components

Tables 6.1–6.3 TF-IDF, AI-assisted, and graph-hybrid comparisons

Table A.1 Repository evidence

Table C.1 Job and profile data dictionary

Table E.1 Test strategy

Table F.1 Risk register

Table V.1 Work packages

Table T.1 System profile

Table S.1 Data-field handling

Table AJ.1 Evidence and interpretation framework

Table AB.1 Change log

Table AA.1 Detailed test cases

Table X.1 Evaluation glossary

Table W.1 Source-to-claim traceability

Table V.1 Future work packages

Table S.1 Job and profile field handling

Table M.1 Requirements traceability

Table J.1 Evaluation reporting

Chapter 1 – Introduction

1.1 Background

Job search involves comparing a person's experience, skills, and interests with vacancies expressed in varied language. Keyword matching is straightforward and reproducible, but may miss related terminology or treat a shared term as sufficient evidence of fit. Semantic representations can capture contextual similarity, while explicit skill overlap can make recommendations easier to explain. A structured occupation and skill graph can represent relationships across jobs and occupational concepts, provided that source mappings are available and quality-controlled.

The project addresses this information-retrieval problem through personalized ranking. The capstone title is "AI-Enhanced Personalized Job Recommendation Platform." CareerIntel is the research extension: a hybrid language-model and knowledge-graph framework for skill-aware recommendation and labour-market analytics. These names refer to related stages of work, not evidence that every research module is already integrated into a deployable application.

1.2 Problem Statement

A job seeker needs a ranked set of opportunities that reflects both profile context and explicit capabilities. A system must normalize profile and job information, handle sparse or inconsistent skill fields, and give useful reasons for a match. The technical challenge is to compare text and skills while maintaining evaluation discipline: recommendation quality cannot be established using unverified labels or a small, selected candidate pool alone.

1.3 Objectives

- Develop and compare TF-IDF and semantic job-ranking baselines.
- Combine semantic, skill-overlap, and job-title signals in a configurable hybrid ranker.
- Normalize skill names and identify missing skills relative to selected job requirements.
- Represent job, occupation, and skill relationships through O*NET-oriented mapping and graph-building scripts.
- Evaluate lexical, semantic, direct-hybrid, and graph-hybrid rankings with top-k metrics, noting the limits of the labels and candidate sets.
- Document the Streamlit and React/FastAPI interfaces separately from the research evaluation pipeline.

1.4 Scope

The repository contains a Streamlit recommendation interface and a separate React/Vite frontend with a FastAPI backend. Both interfaces call src/recommendation/recommend_jobs.py; the backend also stores one career profile in SQLite. The app-facing ranker uses TF-IDF text similarity with skill, requirement, role, and experience scores. The SentenceTransformer HybridRanker and graph-hybrid comparison are separate research modules and are not called by these interfaces. The processed jobs CSV and graph input files are missing, so I could not run the recommendation or market-insights flows end to end. Displayed scores are ranking signals, not probabilities of employment.

1.5 Contributions

The project includes separate lexical, semantic, direct-hybrid, and graph-hybrid ranking approaches, a Streamlit app, a React/FastAPI interface, profile storage, skill-gap analysis, and O*NET mapping scripts. The saved evaluations provide an initial comparison across three profiles. Because their labels are provisional or AI-assisted and the input data are missing, the results should be read as exploratory rather than as a final ranking benchmark.

1.6 Report Organization

Chapter 1 introduces the project. Chapter 2 reviews related research. Chapter 3 defines the problem and requirements. Chapter 4 explains the architecture and ranking methods. Chapter 5 describes the Python modules and application interfaces. Chapter 6 reports saved evaluation results and their limitations. Chapter 7 concludes and outlines future work. The appendices provide repository evidence and technical details.

Chapter 2 – Literature Review

2.1 Recommendation Formulations

Job recommendation can be formulated as ranking candidate vacancies for a query consisting of a seeker profile. Information-retrieval systems commonly use lexical weighting such as TF-IDF, which emphasizes terms distinctive within a corpus. This offers a transparent baseline but depends on surface-form overlap [1]. Dense sentence representations encode text into vectors and can retrieve semantically related descriptions even when wording differs. Their scores are similarity signals rather than calibrated probabilities.

Job-recommender research also emphasizes temporal vacancy behavior, reciprocal interaction between job seekers and employers, fairness, and the dependence of validation on available interaction data [4]. Resume-to-job matching studies illustrate the importance of task-specific labels and annotation protocols; a context-aware transformer study, for example, reports expert annotation and inter-annotator agreement for its clinical-role dataset [6]. These results provide methodological context only and are not direct performance comparisons with CareerIntel.

2.2 Skill-Aware Matching

Skill-aware matching makes capability overlap explicit. A set-based coverage score can identify skills present in a profile and those required by a role. This supports explanation and gap analysis, but depends on skill extraction and normalization. Alias handling can merge known variants (for example, “py” and “python”), yet synonym coverage and context remain difficult. A skill appearing in text does not by itself establish proficiency, recency, or evidence of experience.

2.3 Hybrid Ranking

A hybrid ranker combines signals with different strengths. In the supplied CareerIntel class, semantic similarity, skill coverage, and title relevance are linearly weighted. Such a design is easy to reason about and tune, but the weights are choices, not learned or validated parameters unless a separate experiment supports them. Correlated signals can also double-count evidence.

2.4 Knowledge Graphs and Occupational Taxonomies

A knowledge graph represents entities and typed relations. Job-recommendation research has used graph structures to combine contextual and behavioral relations [5], while CareerIntel links job titles to occupations and occupations to skills. The repository includes scripts for building these mappings and a graph-hybrid scorer. The GitHub results folder also contains saved evaluation-to-occupation matches and graph-hybrid comparison tables. The O*NET source tables and generated graph edge files are absent, so mapping coverage and the saved graph results cannot be independently reproduced.

O*NET provides occupational and skill information through versioned database releases [7]. CareerIntel scripts expect O*NET occupation and skill tables. The repository does not identify the release used to create its saved mappings.

2.5 Evaluation Practice

Precision@k measures the fraction of the first k results judged relevant. Recall@k measures the fraction of all known relevant candidates retrieved in the first k. NDCG@k discounts relevant items lower in the list and normalizes by an ideal ordering; it can accommodate graded gains [3]. Mean reciprocal rank (MRR) averages the reciprocal rank of the first relevant result. NDCG's discounted-gain formulation follows standard graded-relevance evaluation [3]. These metrics depend on the candidate set and relevance judgments. Incomplete or model-generated labels can bias all reported measures.

2.6 Research Gap and Positioning

CareerIntel combines semantic matching and explicit skill signals in its direct hybrid ranker. The repository also contains a graph-aware ranker and saved comparisons among semantic-only, traditional-hybrid, and graph-hybrid configurations. The graph-v3 output reports metrics for these three methods, but the graph hybrid changes both the graph signal and the semantic/direct-skill weights. The comparison therefore does not isolate the effect of the graph component. Missing input files also prevent an independent rerun.

2.7 Summary

The ranking methods provide different signals, but their performance can only be judged reliably with documented data, independent relevance labels, repeatable candidate sets, and an analysis of uncertainty. I use the saved scores in this report as preliminary results.

Chapter 3 – Problem Definition and System Requirements

3.1 Formal Problem

Let u denote a seeker profile and J={j1,…,jn} a candidate job set. The system computes a score s(u,j) for each job and returns jobs sorted in descending order. The score is a ranking signal only. For skill sets Su and Sj, a simple overlap measure is $|Su \cap Sj|/|Sj|$ when Sj is nonempty; the denominator reflects required-skill coverage. When Sj is empty, the implementation must define a fallback rather than divide by zero.

3.2 Stakeholders and Use

The intended user is a job seeker who enters a profile and skills, reviews ranked opportunities, and checks skill gaps. The repository has a Streamlit app and a React/FastAPI interface for this workflow. Evaluation

scripts provide a separate route for comparing ranking methods using candidate profiles, vacancies, and relevance labels.

3.3 Functional Requirements

ID	Requirement	Evidence/scope
FR-1	Accept profile text and skills for ranking	Ranker APIs support profile/query text and skill inputs
FR-2	Rank jobs by lexical similarity	TF-IDF baseline module
FR-3	Rank jobs by semantic similarity	Sentence-transformer ranker module
FR-4	Combine semantic, skill, and title signals	HybridRanker class; experimental module
FR-5	Identify missing skills for target roles	Skill-gap analyzer and saved summaries
FR-6	Map jobs to occupational concepts and skills	O*NET mapping/graph scripts; source files absent
FR-7	Evaluate ranked lists with top-k metrics	Evaluation scripts and saved CSVs
FR-8	Provide an interactive job recommendation application	Streamlit app.py and React/Vite frontend with FastAPI API are present. The job and graph data required for recommendations and market insights are not included, so the complete flow could not be run from this repository.

3.4 Non-Functional Requirements

ID	Requirement	Project interpretation
NFR-1	Reproducibility	Versioned inputs, fixed configuration, candidate snapshots, deterministic ranking where supported
NFR-2	Explainability	Expose score components and matched/missing skills
NFR-3	Data quality	Validate required fields, missing values, IDs, and duplicate records
NFR-4	Privacy and fairness	Document data permissions and assess disparate impacts before real deployment
NFR-5	Performance	Measure latency and memory on a documented dataset; no such benchmark verified
NFR-6	Reliability	Handle empty text, missing skills, invalid fields, and unavailable model assets

3.5 Constraints and Assumptions

The data/raw and data/processed directories contain only placeholders; the job CSV and O*NET/graph input files required by the recommendation and graph workflows are not included. I could not verify the full dataset size, collection dates, source, licence, or graph coverage. The repository does contain saved evaluation and graph-comparison outputs, but the missing inputs mean those results cannot be independently reproduced.

Chapter 4 – System Design and Methodology

4.1 Application and Recommendation Architecture

The repository contains two interface paths. The Streamlit app calls src/recommendation/recommend_jobs.py directly. The React/Vite frontend sends recommendation and profile requests to FastAPI; its recommendation route calls the same recommend_jobs.py function, while the profile route stores fields in SQLite. Both paths need the processed jobs CSV, which is not included. Figure 4.1 summarizes the code connections; it does not show a tested or deployed run.

React/Vite → FastAPI → recommend_jobs.py

Streamlit app.py → recommend_jobs.py

FastAPI profile API ↔ SQLite career_profile table

recommend_jobs.py and market-insights endpoint → processed jobs CSV*

TF-IDF-based recommendations, match components, and skill gaps

Figure 4.1 Source-level application and recommendation workflow (*processed data not included)

4.2 Research-Extension Pipeline

The repository provides separate implementations for TF-IDF, semantic, direct-hybrid, and graph-hybrid ranking. Evaluation scripts prepare candidates, score them, join predictions to the supplied labels, and calculate per-profile and aggregate metrics. The graph-v3 script maps evaluation jobs to occupations and combines semantic, direct-skill, and graph-skill scores. The code and saved outputs show that this comparison was run previously, but the required data and graph inputs are not included for reproduction.

Candidate jobs + seeker profile

↓

TF-IDF | sentence encoder | direct skills | graph skills

↓

Semantic-only | direct hybrid | graph hybrid

↓

Ranked candidates joined to provisional relevance labels

↓

P@10, R@10, NDCG@10, MRR + per-profile diagnostics

Figure 4.2 Evaluation pipeline used in saved ranking comparisons

4.3 Ranking Algorithms

4.3.1 TF-IDF Baseline

The baseline vectorizes text using term frequency–inverse document frequency and computes cosine similarity between a profile representation and job representations. A representative formulation is $tfidf(t,d)=tf(t,d)\cdot\log(N/df(t))$, where N is the number of documents and df(t) is the number containing term t. Cosine similarity is $\cos(x,y)=x\cdot y/(||x||\ ||y||)$. The baseline gives a lexical relevance ordering; the exact saved evaluation pipeline and hyperparameter state should be reproduced from the experiment scripts before interpreting it as a final benchmark.

4.3.2 Semantic Ranker

The semantic module uses SentenceTransformer with the configured sentence-transformers/all-MiniLM-L6-v2 model. It encodes job text and a profile query as normalized vectors and computes cosine similarity by taking their dot product [2]. The repository specifies a model name but does not include a pinned model revision or a fine-tuning run. The input job data are also absent.

4.3.3 Hybrid Ranker

The hybrid ranker computes semantic similarity, profile-to-job required-skill overlap, and title relevance. Its class defaults assign weights ws=0.60, wk=0.30, and wt=0.10, whose sum is one. The combined score is: $S(u,j)=ws\cdot Ssemantic(u,j)+wk\cdot Sskill(u,j)+wt\cdot Stitle(u,j)$. The module comments state that scores are ranking signals, not employment probabilities. The observed source code clips cosine similarity mapped from $[-1,1]$ into $[0,1]$, normalizes common skill aliases, and prepares job text from title, description, responsibilities, qualifications, experience level, and industry when present.

4.3.4 Application Ranking Function

The Streamlit and React/FastAPI interfaces call recommend_jobs.py. This function ranks jobs with TF-IDF content similarity and a weighted score: $S = 0.20C + 0.30K + 0.15Q + 0.30R + 0.05E$, where C is content similarity, K is candidate-skill coverage, Q is job-requirement coverage, R is target-role relevance, and E is experience compatibility. If a component is unavailable, the function renormalizes the weights of the remaining components. It can also filter results by minimum content similarity, require at least one matching skill when skills are provided, and apply an entry-level experience filter. This app-facing function does not call the SentenceTransformer HybridRanker or the graph-hybrid evaluator.

4.3.5 Skill-Gap Analysis

For a selected target job, the analyzer compares normalized required skills with profile skills. The missing set is $G=R\setminus P$, where R is the required-skill set and P is the profile-skill set. A coverage ratio can be expressed as $C=|R\cap P|/|R|$ for $|R|>0$. These quantities describe textual skill coverage only. They do not evaluate proficiency, qualification equivalence, or whether a person can perform a role.

4.4 Knowledge-Graph Schema

Job posting ──mapped_to──> Occupation

Occupation ──requires/uses──> Skill

Occupation ──related_to──> Occupation

Job posting ──mentions──> Skill

Figure 4.3 Knowledge-graph entity and relation schema

The build_graph.py script reads job-to-occupation matches, occupation records, essential/transferable/software skill tables, and related occupations. It writes job, occupation, and skill nodes together with job–occupation and occupation–skill edges. The graph input tables and generated edge files are not in the repository, although saved evaluation outputs include a separate evaluation-job occupation mapping file. This distinction matters: the source defines the graph pipeline, and the results folder preserves some outputs, but the full graph build cannot be repeated from the files provided.

4.5 Evaluation Protocol

The saved evaluation outputs report top-10 metrics for three profiles. The earlier TF-IDF, semantic, and direct-hybrid files use 178 candidate rows; the preliminary AI-assisted review comparison uses 180; the graph-hybrid comparisons use 135. Their label_type fields identify provisional_assistant or preliminary_ai_assisted_review. The metrics therefore describe agreement with those labels on the saved candidate sets, not independently established relevance or user outcomes. The experiment plan does not identify the data snapshot or configuration used for each saved CSV, so I report these files as saved results without assuming they can be reproduced.

Chapter 5 – Implementation

5.1 Technology and Repository Structure

The repository combines Python ranking and evaluation modules with a Streamlit app, a React/Vite frontend, and a FastAPI backend. The Python requirements file lists pandas, NumPy, SciPy, scikit-learn, sentence-transformers, NetworkX, Streamlit, and other data and development packages. The frontend declares React, Vite, Lucide React, and Recharts. FastAPI and Uvicorn are used by the backend source but are not listed in requirements.txt; pyproject.toml does not declare them either, so the documented Python setup is incomplete for the API.

5.2 Implemented Modules and Research Workflow

Component	Observed implementation	What I found
Lexical ranking	TF-IDF vectorization in recommend_jobs.py and the baseline evaluation module.	Both interfaces call recommend_jobs.py, which uses TF-IDF content similarity.
Semantic ranking	SentenceTransformer encoder and cosine ranking in semantic_recommender.py.	This is a separate research module; the interfaces do not call it.
Hybrid ranking	HybridRanker combines semantic, skill, and title signals; semantic_recommender.py includes graph-aware scoring.	These are research/evaluation components. The user interfaces call recommend_jobs.py instead.

Component	Observed implementation	What I found
Skill-gap analysis	Normalize profile/job skills; report gaps	The skill-gap analyzer and its saved summaries are included as separate files.
Graph mapping	Fuzzy/semantic job-to-O*NET mapping and graph table builder	I found the mapping scripts, but not the source data or generated graph files.
Preprocessing	Job cleaning, profile quality, skill normalization	The processing scripts are included, but the dataset is not.
Evaluation	Candidate creation, labeling, review, metric and audit scripts	The project includes evaluation scripts and saved outputs.
Streamlit app	Root app.py accepts profile text, skills, and top-k; displays recommendations and skill gaps.	The Streamlit source calls the recommendation modules directly. Its full run needs the processed jobs CSV, which is not included.
React frontend	Vite/React interface with Overview, Find Jobs, Skill Analysis, My Profile, and Market Insights views.	The frontend calls the FastAPI endpoints. Data-dependent views need the missing processed jobs CSV.
FastAPI backend	Health, profile, recommendation, and market-insights endpoints.	The API calls recommendation code, and profile routes read/write SQLite. FastAPI/Uvicorn are not listed in requirements.txt.
SQLite profile store	career_profile table with a single profile row and profile fields.	Database schema and API code are present. No user-study or production-use evidence is included.

5.3 Data Processing

The code expects job fields such as IDs, titles, descriptions, normalized or parsed skills, responsibilities, qualifications, experience level, and industry in relevant modules. The expected fields differ between scripts, and some loaders check that their required columns are present. The available files do not provide the underlying dataset, so no total job count, profile count beyond evaluation tables, source, collection window, or licensing statement can be responsibly reported.

5.4 Application Interfaces

The root app.py is a Streamlit interface. It accepts profile text and skills, lets the user select how many jobs to return, calls recommend_jobs, displays ranked jobs and match components, and provides a skill-gap analysis for a selected job. The repository also includes a React/Vite interface with Overview, Find Jobs, Skill Analysis, My Profile, and Market Insights views. It calls FastAPI endpoints for recommendations, profiles, and market summaries. The profile API stores one career profile in a SQLite career_profile table. The recommendation and market-insights endpoints depend on data/processed/jobs/jobs_with_normalized_skills.csv, which is not included; I could therefore inspect the application flow in code but could not verify a complete run using the repository data.

5.5 Reproducibility and Setup

The README describes separate commands for starting FastAPI and the React frontend. The repository also includes a Streamlit app, but the README does not document how to launch it. The processed job CSV needed by the recommendation modules is absent, so I could not reproduce the application results or saved experiments from this checkout. I did not run the tests for this report. The Python dependency lists also omit FastAPI and Uvicorn, which need to be installed separately for the API.

Chapter 6 – Results, Evaluation and Discussion

6.1 Available Result Artifacts

The repository contains CSV and JSON outputs for TF-IDF, semantic, direct-hybrid, reviewed, and graph-hybrid comparisons, as well as ranking audits and skill-gap summaries. The tables below transcribe three saved comparisons. The corresponding label files identify provisional assistant or preliminary AI-assisted judgments. Since the input job and O*NET data are missing, I could not reproduce the evaluations from this checkout.

6.2 Provisional Assistant-Labeled Comparison

Model	Profiles	Candidates	P@10	R@10	NDCG@10	MRR
TF-IDF	3	178	0.6333	0.3368	0.8616	0.8333
Semantic	3	178	0.7333	0.4082	0.9211	0.8333
Hybrid	3	178	0.7333	0.4082	0.9531	1.0000

Table 6.1 Saved metrics labeled “provisional_assistant” in the evaluation outputs. Values are means across three profiles. The hybrid row has the highest NDCG@10 and MRR in this particular output, while semantic and hybrid tie on precision and recall. Since the labels are provisional and assistant-generated and the sample is three profiles, this is an exploratory result only. It cannot establish independent ranking quality or general superiority.

6.3 Preliminary AI-Assisted Reviewed Comparison

Model	Profiles	Candidates	P@10	R@10	NDCG@10	MRR
TF-IDF	3	180	1.0000	0.3196	1.0000	1.0000

Semantic	3	180	1.0000	0.3196	1.0000	1.0000
Hybrid	3	180	1.0000	0.3196	1.0000	1.0000

Table 6.2 Saved metrics from reviewed_model_comparison, labeled “preliminary_ai_assisted_review.” All three systems have identical aggregate metrics in this file. This could reflect ties in the judged candidates or a limitation in the candidate/label set; it is not evidence of equivalence in broader settings. The word “reviewed” in the directory name does not establish independent human adjudication: the stored label_type explicitly says AI-assisted review.

6.4 Graph-Hybrid Comparison

Model	Profiles	Candidates	P@10	R@10	NDCG@10	MRR
Semantic-only	3	135	0.8000	0.5119	0.8451	1.0000
Traditional hybrid	3	135	0.8333	0.5264	0.8702	1.0000
Graph hybrid	3	135	0.8333	0.5264	0.8699	1.0000

Table 6.3 Saved graph-v3 comparison; label_type is provisional_assistant. Means are reported for three profiles across 135 candidate rows.

6.5 Metric Interpretation

Precision@10 divides relevant items in the first ten positions by ten. Recall@10 divides relevant items in the first ten by the total known relevant items for that profile. NDCG@10 discounts lower-ranked gains logarithmically and divides the observed discounted gain by the ideal gain. MRR is the mean reciprocal of the rank of the first relevant result. Their meaning is conditional on how relevance was labeled and which jobs were eligible candidates. Candidate generation can exclude relevant jobs, and a small or constructed candidate pool can inflate or compress scores.

6.6 Skill-Gap and Graph Evidence

The repository contains saved skill-gap summaries, graph-mapping scripts, and graph-hybrid evaluation outputs. The graph-v3 summary covers three profiles and 135 candidate rows. Semantic-only scores are P@10 0.8000, R@10 0.5119, NDCG@10 0.8451, and MRR 1.0000. The traditional hybrid scores 0.8333, 0.5264, 0.8702, and 1.0000; the graph hybrid scores 0.8333, 0.5264, 0.8699, and 1.0000. The graph-hybrid formula uses 0.68 semantic, 0.17 direct-skill, and 0.15 graph-skill weights, while the traditional hybrid in that comparison uses 0.80 semantic and 0.20 direct-skill weights. Since more than the graph signal changes, this is not a controlled graph ablation. All labels are provisional_assistant, and the missing job/O*NET/graph inputs prevent independent reproduction. The skill-gap output files also lack their underlying dataset here.

6.7 Limitations and Threats to Validity

Threat	Implication
Provisional or AI-assisted judgments	Potential circularity, label noise, and evaluator bias; not

	independent ground truth
Only three profiles in aggregate comparisons	Very limited profile diversity and weak generalizability
Small, bounded candidate pools	Recall and ranking metrics depend on selection and completeness
Missing raw/processed data	Counts, provenance, licensing, fields, preprocessing outcomes cannot be audited
No verified independent human agreement	Inter-rater reliability and adjudication quality unknown
Graph result cannot be isolated or reproduced	Saved graph scores exist, but graph inputs are absent and the compared models use different weights.
No user-study or deployment evidence	The project files do not show whether users adopted the system or whether it changed search time, satisfaction, or employment outcomes.
Missing run configuration and version details	Without run metadata, it is not possible to confirm that the saved results came from the current code and configuration.
No end-to-end runtime check	Both interface code paths are present, but the processed job dataset is missing, so recommendations could not be run from this checkout.

6.8 Discussion

The saved outputs show small differences among the rankers, but they do not establish which method works best. In the graph-v3 comparison, the graph hybrid has a slightly lower NDCG@10 than the traditional hybrid (0.8699 versus 0.8702), while both have the same Precision@10, Recall@10, and MRR. Another saved graph configuration reports different results, which reinforces the need to pin the exact run and input files. All graph metrics use provisional assistant labels and only three profiles. The source dataset and graph inputs are not included, so these results should be treated as exploratory.

Chapter 7 – Conclusion and Future Work

7.1 Conclusion

The project includes a Streamlit app and a React/Vite interface backed by FastAPI and SQLite profile storage. Both interfaces call recommend_jobs.py, which uses TF-IDF content similarity with skill, requirement, role, and experience signals. Separate source modules implement SentenceTransformer and graph-hybrid research ranking. The processed job data and graph inputs needed to run these components are not in the repository.

The repository includes saved comparisons for three profiles, including TF-IDF, semantic, direct-hybrid, and graph-hybrid methods. Some tables use provisional assistant labels, while another uses preliminary AI-assisted review labels. The results are useful for examining how the ranking components behave, but they are not independent ground truth and do not identify a definitive best model. The project also contains source code for interactive applications; because the job data and graph inputs are absent, the full recommendation flow could not be reproduced from this repository.

7.2 Future Work

- Restore and document the source datasets, including collection dates, field definitions, missingness, deduplication, permissions, and license.
- Run the Streamlit and React/FastAPI interfaces against a versioned job dataset, check the complete profile-to-recommendation flow, and document error handling.
- Create a versioned candidate benchmark with independent human judgments, documented relevance rubric, multiple annotators, adjudication, and inter-rater agreement.
- Expand the profile set and job pool; report confidence intervals or bootstrap uncertainty and assess performance across experience levels and occupational groups.
- Repeat the graph-hybrid comparison with the same data, candidate set, labels, and non-graph weights as the direct hybrid so that the graph contribution is isolated; also report mapping precision and coverage.
- Evaluate skill-gap recommendations with a documented masked-skill protocol and appropriate baselines; distinguish textual absence from lack of proficiency.
- Measure latency, resource use, calibration if scores are interpreted probabilistically, fairness, privacy, and usability before considering deployment.
- Record exact software versions, model revisions, random seeds, input hashes, and experiment commands for reproducibility.

7.3 Closing Statement

CareerIntel combines language-model representations with explicit skill signals and an O*NET-oriented graph component. The repository also includes user-interface code for both Streamlit and React/FastAPI. The next step is to supply versioned job and graph data, run the interfaces end to end, and evaluate the ranking methods with independent relevance judgments.

References

[1] G. Salton and C. Buckley, “Term-weighting approaches in automatic text retrieval,” Information Processing & Management, vol. 24, no. 5, pp. 513–523, 1988. doi:10.1016/0306-4573(88)90021-0.

- [2] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,” in Proceedings of EMNLP-IJCNLP, pp. 3982–3992, 2019. doi:10.18653/v1/D19-1410.
- [3] K. Järvelin and J. Kekäläinen, “Cumulated gain-based evaluation of IR techniques,” ACM Transactions on Information Systems, vol. 20, no. 4, pp. 422–446, 2002. doi:10.1145/582415.582418.
- [4] C. de Ruijt and S. Bhulai, “Job Recommender Systems: A Review,” arXiv:2111.13576, 2021.
- [5] W. Shalaby et al., “Help Me Find a Job: A Graph-based Approach for Job Recommendation at Scale,” arXiv:1801.00377, 2018.
- [6] C. Li, E. Fisher, R. Thomas, S. Pittard, V. Hertzberg, and J. D. Choi, “Competence-Level Prediction and Resume & Job Description Matching Using Context-Aware Transformer Models,” in Proceedings of EMNLP, pp. 8456–8466, 2020. doi:10.18653/v1/2020.emnlp-main.679.

[7] National Center for O*NET Development, “O*NET Database,” O*NET Resource Center. https://www.onetcenter.org/database.html. Accessed 10 November 2026.

[8] Harsha767, “CareerIntel,” GitHub repository, commit ab0b370a5c3cf1de5a3b7a2cd7411be9bd45dca0, 11 November 2026. https://github.com/Harsha767/CareerIntel.

Appendix A – Repository Evidence and Reproducibility

Evidence inspected	Verified observation
README.md and research plan	README describes React/Vite and FastAPI setup; app.py also contains a Streamlit interface. Research plan and results show exploratory experiments.
configs/base.yaml	Seed 42; evaluation k values 5, 10, 20; embedding model name configured
requirements.txt	Python data and ranking dependencies include Streamlit; FastAPI and Uvicorn used by backend are not listed.
data/raw, data/processed	Only .gitkeep placeholders are present; jobs and O*NET source tables are absent.
results aggregate CSVs	Saved 178- and 180-candidate comparisons plus 135-candidate graph-hybrid comparisons; labels include provisional_assistant and preliminary_ai_assisted_review.
src/knowledge_graph	O*NET mapping and graph scripts are present; source tables are absent. Saved graph-comparison outputs include evaluation occupation mappings.
Application source	Root app.py contains a Streamlit recommendation and skill-gap interface; frontend/ and backend/ contain React/Vite and FastAPI code.
GitHub source snapshot	main branch commit ab0b370a5c3cf1de5a3b7a2cd7411be9bd45dca0, inspected 11 November 2026.

The technical description and reported metrics correspond to the supplied CareerIntel source archive and saved result files. The raw vacancy and O*NET input files were not part of that archive, so dataset provenance and processing totals are not stated. The report presents the implementation and experimental outputs within that evidence boundary.

Appendix B – Metric Definitions

Metric	Definition used in report	Interpretation
Precision@k	Relevant items among top k divided by k	Top-list purity
Recall@k	Relevant items among top k divided by all known relevant candidates	Known relevant coverage
DCG@k	Sum of (2^rel−1)/log2(rank+1) up to k	Position-discounted relevance gain
NDCG@k	DCG@k divided by ideal DCG@k	Normalized ordering quality for the judged candidate set
MRR	Mean of 1/rank(first relevant item)	How early the first judged relevant item

		appears

For binary labels, rel is 1 for judged relevant and 0 otherwise. For graded relevance, rel may take ordered gain values. Metric implementation details should be checked against the chosen evaluation script because repository scripts may differ in how they define candidate universes and gain values.

Extended Technical Chapters and Appendices

This part develops the design, implementation, evaluation, and governance analysis in greater depth. It distinguishes implemented methods from their evaluation evidence and identifies the conditions needed to interpret the results responsibly.

Chapter 1 – Introduction (Extended Context)

1.7

Motivation and Context

Online vacancy information is heterogeneous. The same competency can be expressed by a formal name, an abbreviation, a tool name, or a phrase embedded in a responsibility. A profile may be a concise list while a vacancy is a long narrative. Ranking therefore requires choices about representation, normalization, similarity, missing information, and tie handling. These are engineering choices with direct consequences for what appears near the top of a recommendation list.

For students and early-career users, a useful system should expose why a role was retrieved and which requirements are not represented in the current profile. Such explanations should remain appropriately modest. A string-level match can indicate that a term occurs in both documents; it cannot verify a user’s proficiency or certify that a vacancy is suitable. Career guidance also needs human judgment because preferences, location constraints, work authorization, salary expectations, and accessibility requirements are not all reducible to a text score.

The motivation for CareerIntel is to investigate whether a semantic representation can complement lexical matching and whether explicit skill and occupational relations can make retrieval more useful for sparse profiles. This motivation leads to research questions rather than a claim of demonstrated labor-market impact. The repository research plan asks whether embeddings improve NDCG, whether graph-derived relations improve recall for sparse profiles, and whether skill-gap predictions outperform a frequency baseline. The plan also lists these experiments as to-do, so the report treats them as intended questions and not completed results.

1.8

Research Questions and Hypotheses

RQ1 asks whether semantic embeddings improve ranking over a lexical baseline on a documented relevance benchmark. A valid answer requires a fixed candidate set, independent labels, consistent profile inputs, and an evaluation split that prevents tuning on the reported test judgments.

RQ2 asks whether graph-derived skill relations improve recall for profiles with sparse skill information. The repository now includes graph-hybrid result files for three profiles, but the labels are provisional and the input graph files are missing. A controlled answer would still require the same candidate set and labels with only the graph signal changed.

RQ3 asks whether skill-gap predictions outperform a frequency-only baseline. To answer it, the project would need held-out or masked skills, a baseline implementation, and a declared metric. A gap summary file alone does not establish predictive validity.

RQ4 considers simulated feedback for ranking. Simulated clicks or applications are not equivalent to real user feedback and would need a simulation assumption and sensitivity analysis. No adaptive feedback learner is verified in the current source archive.

1.9

Intended Use and Boundaries

The intended role of the prototype is decision support: organize candidate jobs for review and make profile-job overlaps more inspectable. It should not automatically reject candidates, represent rank as probability of hiring, infer protected attributes, or present an incomplete skill list as an objective employability judgment.

If future deployment is considered, product language should distinguish “textual match” from “qualified” or “recommended by an employer.” Users should be able to correct their profile, inspect the job source and date, and understand why a role appeared. Employers and job seekers have asymmetric stakes, so any use for screening requires additional validation, governance, and human oversight.

Chapter 2 – Literature Review (Extended Methodological Review)

2.8

Content-Based Retrieval

Content-based recommendation represents each item by its attributes and compares those attributes with a user representation. In job search, the item representation can combine title, description, responsibilities, qualifications, experience, and explicit skills. The user representation may contain skills and a free-text summary. The advantage is that recommendations can be generated without a large interaction history. The limitation is dependence on profile completeness and on an adequate match between profile and job vocabulary.

A TF-IDF vectorizer maps text into a sparse space. Terms that occur frequently in one document but less widely across the corpus receive greater weight. A cosine score is then used to compare vectors. This is computationally practical for sparse text and offers inspectable term contributions. It does not understand that two distinct words may refer to the same competency unless normalization or additional semantic resources are used.

BM25 is a related lexical retrieval method that accounts for term frequency saturation and document length. The project dependencies list rank-bm25, but a listed package is not proof that a specific saved experiment used it. This report therefore describes TF-IDF where code and outputs confirm it, and treats BM25 as an available dependency or potential baseline only unless a run artifact verifies otherwise.

2.9

Dense Semantic Retrieval

Sentence encoders map a sequence of words into a fixed-dimensional vector. With normalized embeddings, a dot product equals cosine similarity. Dense retrieval can reduce exact vocabulary dependence, but its behavior depends on the model’s training domain, text length, pooling strategy, model revision, and input construction. A similarity score does not necessarily represent relevance or confidence.

The project's configured model name is all-MiniLM-L6-v2 from the sentence-transformers family. The code calls a SentenceTransformer encoder and ranks using normalized representations. The archive does not pin a model artifact checksum or show a training run. Thus the report describes use of a pretrained encoder through the library interface and does not claim that the model was fine-tuned on job data.

2.10

Hybrid Signal Combination

Linear score fusion provides a compact way to combine signals on comparable scales. Before fusion, component ranges and direction must be checked. In the hybrid source, semantic similarity is mapped from cosine range [−1,1] into [0,1], and weights are validated as nonnegative and summing to one. Skill overlap and title relevance are additional signals. If one component has a different distribution, nominal weights alone do not guarantee comparable influence.

An ablation study would compare the full ranker with versions that remove one signal at a time. Weights should be selected on training/development judgments, not on the final test set. Reporting sensitivity across plausible weights can reveal whether conclusions depend on a single hand-chosen configuration. The saved result tables do not include a verified sweep or confidence intervals.

2.11

Taxonomies, Ontologies, and Graph Representations

An occupational taxonomy contributes standardized occupation identifiers and descriptions. A graph adds typed edges that can connect vacancies, occupations, skills, and related occupations. This representation can support propagation from a profile skill to occupations that use related skills, but propagation may also introduce noisy or overly broad matches.

The repository scripts indicate an intended O*NET-oriented pipeline. They refer to occupation master data, essential/transferable/software skill tables, related occupations, and semantic job matches. The graph builder creates node and edge tables and includes quality-check logic. Because neither the inputs nor the graph outputs are included, the scope here is a code-level architecture description, not a report of taxonomy coverage or graph accuracy.

Any future citation of O*NET content should identify the specific release, file names, access date, and applicable terms. Those details cannot be reconstructed from this archive, so no external data source attribution is asserted in this report.

2.12

Fairness, Transparency, and Human Factors

Recommendation systems can reproduce historical patterns in job postings or user interactions. Even a content-based system may rank opportunities differently when profiles use different language, omit nontraditional experience, or describe equivalent skills using different terms. A skills graph can improve synonym coverage but can also privilege skills that are better represented in the source taxonomy.

Fairness assessment requires choosing protected and relevant groups, defining appropriate outcome measures, and ensuring that collection and analysis are lawful and ethically justified. No demographic

dataset, fairness assessment, or user study is present in the supplied archive. The report therefore states fairness as a risk and future requirement, not as a feature or achieved property.

Explanations should communicate evidence actually used by the ranker, such as matched skill terms or text similarity components. They should not infer causal reasons or market demand from a ranking score. A user-facing explanation should disclose missing job fields and allow the user to correct profile information.

Chapter 3 – Requirements and Analysis (Extended)

3.6

User Stories and Acceptance Criteria

Job seeker story: as a user, I can provide a skills list and career summary so that the system can return a ranked set of vacancies. Proposed acceptance criterion: for a valid nonempty profile and a nonempty candidate dataset, the ranking returns at most k distinct job identifiers in descending score order and reports the component signals available for each result.

Job seeker story: as a user, I can inspect skills found in the role and skills not represented in my profile. Proposed acceptance criterion: the system labels these items as text-derived overlaps and gaps, handles empty skill lists without failure, and does not state that a missing term proves lack of competence.

Researcher story: as an evaluator, I can reproduce a comparison using a fixed candidate set, labels, and model configuration. Proposed acceptance criterion: every run records input hashes, code revision, environment, model identifier/revision, seed, candidate-generation method, labels, and metric definitions.

Evaluator story: as a reviewer, I can see which relevance judgments are provisional, AI-assisted, independently human-labeled, or adjudicated. Proposed acceptance criterion: label provenance is preserved in outputs and reported alongside every aggregate metric.

3.7

Data Quality Requirements

Each job record should have a stable unique identifier. Duplicate postings should be detected using source identifiers where available and content-based checks where necessary. Deduplication rules must be documented because identical text can refer to separate locations or postings, while near-duplicates may represent reposts.

Text inputs should distinguish missing values from literal strings such as “None” or “nan.” The project loaders and cleaning functions handle some missing values, but a production-quality pipeline should document field-specific rules and count how many records are dropped or imputed at each stage.

Skills should be parsed without splitting multiword terms incorrectly. Normalization should preserve the original string alongside its canonical form so that users can see the source wording. Alias dictionaries should be versioned and reviewed; automatic fuzzy matches should retain confidence and allow manual audit.

Every data transformation should be traceable. A processing manifest should contain input paths, source identifiers, checksums, schema version, row counts before and after filtering, transformation timestamps, and output hashes. These elements are recommended controls and are not verified outputs in the supplied archive.

3.8

Security and Privacy Requirements

Job seeker profiles may contain personal information. A future service should minimize collection, define retention periods, protect data in transit and at rest, and limit access to authorized roles. The current archive does not include a deployed service or privacy policy, so no operational control is claimed.

Users should be told whether profile text is stored, used for model improvement, shared with third parties, or retained after account deletion. If external model APIs are introduced, their data handling and contractual terms must be reviewed. The current configured sentence-transformer name alone does not establish whether inference is local or hosted in any deployed environment.

For reproducible research, synthetic or de-identified profile examples should be used where feasible. Re-identification risk must be considered when sharing detailed job histories or rare skill combinations.

3.9

Use Case Descriptions

UC-1 Rank jobs: the researcher loads a profile and candidate jobs; validation checks required identifiers and text; the selected ranker produces ordered scores; the output is saved with configuration metadata. Exception paths include empty candidate sets, invalid profile text, duplicate IDs, and unavailable model weights.

UC-2 Analyze skill gaps: the user or evaluator selects a job; profile and job skills are parsed and normalized; intersection and difference sets are calculated; the output reports matched and absent skill strings. Empty and malformed fields should produce explicit “not available” states rather than an incorrect zero-skill interpretation.

UC-3 Evaluate a ranker: a fixed profile-job candidate file and relevance labels are joined to ranked predictions; metric calculations are performed per profile; aggregates and diagnostics are exported. Missing pairs or labels should be counted and reported because omission can affect recall and NDCG.

UC-4 Map to occupation: a posting title or text is matched to an occupational record by a selected method; match scores and identifiers are retained; low-confidence results are flagged for review. The source includes matching scripts, and the results folder includes an evaluation occupation-mapping output; the O*NET and graph input tables are absent.

Chapter 4 – Methodology (Extended Design and Algorithms)

4.6

End-to-End Data Flow

The intended pipeline starts with source records and profile descriptions. A validation stage checks column availability, types, identifiers, and missingness. Text fields are cleaned and selected for the ranker. Skills are

parsed from list-like or delimited values and normalized through alias rules. These steps are visible across repository modules, although a complete run cannot be reproduced because the datasets are absent.

The lexical path vectorizes profile and job text and computes a similarity score. The semantic path encodes job text and the profile query, normalizes vectors, and uses a dot product. The hybrid path computes its component signals and fuses them. Rank outputs are joined to evaluation labels and summarized. Graph mapping is a separate preparation branch whose integration into the ranking path is unverified.

Inputs should be snapshotted before evaluation. If candidate generation changes between models, comparisons are confounded. If one model has access to fields another does not, the experiment measures both model and information availability. A controlled comparison should hold input fields and candidate IDs fixed.

4.7

Text Construction and Normalization

The hybrid ranker constructs job text from fields including title, job_description, job_text, responsibilities, minimum_qualifications, preferred_qualifications, experience_level, and industry when present. It also appends normalized skills when available. This design provides richer context, but it may repeat content if fields overlap. Repetition can change vector weighting and should be audited.

The skill parser accepts iterable values, Python literal list strings, and delimited text. The alias examples include py → python, ml → machine learning, data visualisation → data visualization, and scikit learn → scikit-learn. These are implementation details from the ranker. They do not constitute comprehensive skill ontology coverage.

Case folding and whitespace normalization reduce superficial variation. Punctuation removal, stemming, lemmatization, and language detection should be applied only with a documented rationale, because they can damage product names, acronyms, or multiword technical terms. The archive contains skill normalization and text cleaning utilities but does not demonstrate a full multilingual processing workflow.

4.8

Detailed Ranking Procedure

- Step 1: Validate that candidate jobs exist and that each usable job has an identifier and title where required by the ranker.
- Step 2: Build each job text from the configured descriptive fields and append normalized skill strings where available.
- Step 3: Parse and normalize required skills for each job and profile skills for the query.
- Step 4: Encode job texts and profile text with the selected lexical or sentence representation.
- Step 5: Compute semantic similarity and skill/title signals as defined by the chosen model.
- Step 6: Combine signals using configured weights only in the hybrid model; preserve component values.
- Step 7: Sort by descending combined score, apply stable tie-breaking, and return the requested top-k set.
- Step 8: Persist model name, parameters, candidate IDs, run metadata, and score components with the output.

4.9

Score Interpretation and Calibration

Ranking requires only an ordering; calibration is a separate property. A score of 0.8 does not imply an 80 percent probability of suitability or hiring. The hybrid module explicitly describes outputs as ranking signals rather than probabilities. If a future interface displays bars or percentages, the labels must avoid probabilistic interpretation unless calibration is demonstrated on a relevant dataset.

Cosine similarity is bounded and normalized in the semantic implementations, but this does not guarantee comparable relevance across users or domains. Score distributions can vary with profile length, domain-specific vocabulary, and candidate composition. Per-profile analysis is therefore more informative than a single aggregate alone.

4.10

Graph Construction and Quality Controls

The graph builder’s design includes stable job IDs, job nodes, occupation nodes, skill nodes, and relation tables. Job-to-occupation edges derive from semantic matches; occupation-to-skill edges come from essential, transferable, and software skill files; related-occupation edges represent occupational relations. Quality checks in the builder are intended to detect structural issues such as missing required columns and invalid references.

Graph quality should be measured at multiple levels: node coverage, edge coverage, duplicate rate, dangling references, mapping confidence, and manual precision on an audited sample. A graph may be syntactically valid yet semantically wrong. Low-confidence title mappings should not be treated as authoritative occupation assignments.

If graph evidence is incorporated into ranking, the propagation rule must be specified. For example, a job may receive additional score when its mapped occupation connects to profile skills through a defined edge type. Path length, edge weights, direction, and confidence threshold affect results. The graph scorer and saved graph-hybrid evaluation are present, but the input graph files are missing and the comparison changes multiple weights.

4.11

Evaluation Design Recommendations

The benchmark should separate profile construction, candidate generation, relevance annotation, and ranking. Annotators should see enough job context to judge relevance and should use a rubric with graded categories. The annotation interface should avoid showing model identity or rank if the purpose is blind relevance judgment.

Candidate pools should include realistic hard negatives and relevant roles that may not share exact keywords. The pool should be frozen before evaluating all models. When pooling model outputs, relevance judgments should include candidates contributed by each model to reduce evaluation bias toward one system.

Profile-level metrics should be reported before aggregate means. Confidence intervals can be estimated by resampling profiles, but three profiles are insufficient for stable inference. Statistical tests should respect paired per-profile outcomes and avoid treating each job candidate as an independent user observation.

For sparse profiles, define sparsity operationally, such as a limited count of explicit skills or a missing summary, then report subgroup results. A graph benefit claim requires a predeclared comparison and enough sparse profiles to estimate uncertainty.

Chapter 5 – Implementation (Extended Module Walkthrough)

5.6

Package-Level Components

The newer src/careerintel package contains data audit utilities, metric helpers, and model classes. Its model modules are intentionally compact abstractions: baselines.py exposes a TF-IDF ranker, semantic_ranker.py wraps SentenceTransformer encoding, and hybrid_ranker.py combines semantic, skill, and title signals. These modules are distinct from the larger src/recommendation scripts, which contain project-specific processing and file conventions.

The src/recommendation directory contains recommend_jobs.py, semantic_recommender.py, skill_gap_analyzer.py, and learning_recommender.py. The Streamlit app in app.py and the React/Vite frontend call the recommendation functionality through separate paths. The processed job dataset needed by both paths is not included, so their source-level connections are visible but the complete recommendation flow cannot be run from this repository.

The src/preprocessing directory contains job cleaning and profile quality scripts. src/nlp includes skill audit and normalization scripts. src/analysis contains exploratory job analysis. src/ingestion includes a job-download script. The presence of an ingestion script does not establish successful acquisition, provenance, or permission to use any retrieved data.

5.7

Evaluation Utilities

The evaluation directory includes scripts for creating evaluation datasets, balancing candidates, generating candidate labels, reviewing labels, running TF-IDF/semantic/hybrid comparisons, running independent comparisons, generating diagnostics, and auditing top-10 candidates. This breadth supports a research workflow, but outputs must be read together with their label provenance and candidate construction logic.

The repository includes tests for baseline behavior, metrics, skill normalization, and skill-gap analysis. Their presence is verified, but they were not executed as part of this report. Test files are not evidence that the delivered environment passes them or that the complete data pipeline is integration-tested.

The saved outputs include ranked recommendations, per-profile metrics, model summaries, top-10 overlap, missing reviewed pairs, and audited top-10 candidates. These are useful for diagnostic review, but the raw inputs and run logs are absent, preventing full reproduction.

5.8

Input and Output Contracts

A robust job input contract should require a stable job identifier and a title, permit optional description and qualification fields, and represent skills as either structured arrays or explicitly delimited text. A profile input contract should separately capture summary text and structured skills. Optional location, experience, compensation, and work mode should be treated as explicit constraints if the system uses them.

Ranker output should include job identifier, rank, total score, component scores, matched skills, missing skills, source posting fields, and a clear score semantics note. The existing hybrid implementation returns component scores; the exact presentation or persistence contract of an integrated app is not verified.

Evaluation output should include profile identifier, model identifier, candidate set identifier, per-item rank and relevance, per-profile metrics, aggregate metrics, and label type. The result CSVs include model/profile/candidate summary fields, but a complete experiment manifest is absent from the supplied archive.

5.9

Error Handling and Operational Considerations

Potential errors include missing files, absent required columns, invalid list-like skill strings, duplicate job identifiers, empty profile text, unavailable model weights, memory pressure during batch embedding, and incomplete relevance joins. Some source modules perform validation and raise errors for empty input or invalid identifiers. A full operational error-handling audit would require executing the system with representative inputs.

Embedding inference can be batched to control memory. Corpus embeddings may be cached with a key that includes model revision and text preprocessing version. Cache invalidation matters: changed job text should not reuse stale vectors. The repository contains no deployment configuration or performance benchmark.

5.10

Deployment Readiness Assessment

I could not assess production readiness. The repository contains application source code, but no deployment configuration, load test, monitoring setup, or dataset source record. The job data needed for an end-to-end run are also missing.

Before a pilot, the project would need documented source permissions, secure profile handling, repeatable model loading, user-visible data freshness, application-level integration tests, accessibility review, and a process to report inaccurate or harmful recommendations. These are proposed readiness gates.

Chapter 6 – Evaluation (Extended Analysis)

6.8

Metric Calculation Details

For binary relevance labels, Precision@k is the count of relevant results among the first k divided by k. If fewer than k results are available, the evaluation implementation may still divide by k; the exact convention should be confirmed in each script. Recall@k divides the relevant top-k count by the total relevant items in the candidate set. If no candidate is labeled relevant, recall is undefined; implementations may return NaN or apply another documented rule.

DCG@k accumulates graded gain divided by a logarithmic rank discount. The evaluated script uses gain (2^rel−1) and discount log2(rank+1). NDCG normalizes actual DCG against an ideal ordering formed by sorting the full candidate relevance array and truncating to k. This makes the candidate universe part of the metric definition.

MRR is reciprocal rank of the first relevant result for each profile, averaged across profiles. It emphasizes the first hit and ignores the position and number of additional relevant items after that first hit. Therefore it should be interpreted alongside recall and NDCG rather than as an overall quality score.

6.9

Reading the Provisional Results

In the provisional assistant-labeled CSV, the TF-IDF model reports mean precision 0.6333, recall 0.3368, NDCG 0.8616, and MRR 0.8333. The semantic output reports 0.7333, 0.4082, 0.9211, and 0.8333, respectively. The hybrid output reports 0.7333, 0.4082, 0.9531, and 1.0000. These values are transcribed from the saved aggregate artifacts and should not be interpreted beyond their narrow labeled set.

The metric values can be mathematically correct relative to the labels and still fail to estimate real-world relevance. If the labels were generated with heuristics or an assistant influenced by the same text signals used by the ranker, the evaluation may favor those signals. The label_type field explicitly marks this comparison provisional assistant-generated, making that caveat central rather than incidental.

The reviewed-model-comparison CSV reports all three models with precision 1.0, recall 0.3196, NDCG 1.0, and MRR 1.0 over 180 candidates. Identical values can occur if the top-ten set and the first relevant item align in the same way for each model, even while lower-ranked ordering differs. The aggregate table alone cannot establish that rankings are identical; the per-item file would need to be analyzed with a documented protocol. The label type remains preliminary AI-assisted review.

6.10

Candidate Set Effects

Recall uses the number of known relevant jobs in the evaluated candidate set. If the candidate-generation stage excludes a suitable posting, no model can retrieve it and the denominator omits it. If candidates are selected from model top results, the benchmark can overrepresent items that one or more systems are already good at retrieving.

Candidate counts of 135, 178, and 180 appear in different saved comparisons, but count alone does not establish benchmark quality. The selection process, realism of alternatives, relevance judgments, and profile coverage also matter. The source datasets are not in the repository, so I could not reconstruct these candidate pools.

6.11

Label Provenance and Bias

Label provenance should be represented at the item level. A label can be machine-suggested, assistant-generated, human-assigned, adjudicated, or unknown. Mixing these statuses into a single “relevant” flag hides the uncertainty. The result files disclose aggregate label types, but the present report cannot verify the independence or qualifications of any human reviewer.

AI-assisted labeling can accelerate annotation but may introduce correlated judgment errors. An independent study would sample a subset for multiple human annotators, measure agreement using a suitable statistic, investigate disagreements, and adjudicate using a prespecified rubric. The candidate labels should be blinded to model identity and rank where possible.

6.12

Statistical Reporting Recommendations

With three profiles, a mean can be dominated by one profile and provides little information about between-user variation. A full report should include each profile metric, sample size, variance or interval where appropriate, and a transparent account of missing labels. Profile-level paired differences are the natural unit for comparing rankers when every ranker is evaluated on the same profiles and candidates.

Bootstrap intervals should resample at the profile level rather than at the candidate-row level when the target population is users. Three profiles are too few for a reliable bootstrap approximation, so the current artifacts should remain descriptive. No significance or confidence claim is made in this report.

6.13

Relevance Rubric Proposal

A future relevance rubric could assign graded judgments: 0 for unrelated, 1 for weak or adjacent match, 2 for plausible match with important gaps, and 3 for strong textual alignment with stated profile evidence. The rubric should distinguish role-family similarity, skill match, seniority, location and work constraints, and user preference. The final score or decision rule should be defined before annotation.

Annotators should receive a short handbook with examples, edge cases, and instructions for missing information. A “cannot judge” option is necessary when job descriptions are incomplete. Annotators should not infer protected characteristics or assume that missing profile evidence means a user lacks a skill.

6.14

Qualitative Failure Modes

Vocabulary mismatch: a profile says “data visualization” while a role uses a product-specific name or an alternative spelling. Alias normalization may help only if the mapping is known.

Overlong job descriptions: generic corporate language may dominate a semantic representation and dilute role-specific requirements. Field selection and text truncation strategy should be documented.

Skill list noise: extracted or manually entered skills may contain broad terms, duplicates, or ambiguous strings. Set overlap can overcount weak matches unless canonicalization and context are handled carefully.

Cold start: a profile with little information gives the ranker limited evidence. A system should ask for clarifying preferences or explain uncertainty rather than imply precision.

Outdated vacancy: a high match to a closed or stale posting is not useful. A future application should verify posting freshness and preserve source timestamps.

Occupation mapping error: a title with multiple meanings may be mapped to an inappropriate occupation. Confidence thresholds and human review are needed for consequential use.

6.15

Results Summary

The saved files contain ranking and diagnostic outputs. They do not measure user satisfaction, employment outcomes, market trends, or the effect of graph features. Further experiments will need documented data and stronger evaluation methods.

Chapter 7 – Responsible Development and Future Work

7.4

Phased Research Roadmap

Phase 1 – Data recovery and documentation: locate the exact source dataset, verify its provenance and terms, create a schema and datasheet, and reproduce preprocessing counts. If data cannot be redistributed, document a secure reproducibility procedure and share derived artifacts only when allowed.

Phase 2 – Benchmark construction: create fixed profile-job candidate sets, build an annotation rubric, collect independent judgments, assess agreement, and adjudicate disagreements. Freeze a held-out test set before model tuning.

Phase 3 – Baseline re-evaluation: run TF-IDF and semantic rankers on identical text fields and candidates. Record model revision, seed, environment, and detailed results. Add BM25 only if implementation is verified and controlled.

Phase 4 – Hybrid ablations: evaluate semantic, skill, title, and combined variants. Tune weights on development data, conduct sensitivity analysis, and report per-profile results with uncertainty where the sample supports it.

Phase 5 – Graph experiment: build and audit the graph, establish mapping precision, define a graph scoring rule, and compare against an otherwise identical ranker. Report both overall and sparse-profile subgroup outcomes.

Phase 6 – Application validation: run the existing Streamlit and React/FastAPI interfaces with the documented data contract. Check the full profile-to-recommendation flow, missing-data errors, accessibility, privacy, and usability.

Phase 7 – Controlled pilot: if appropriate and approved, conduct a user study with a defined protocol and participant consent. Measure perceived usefulness and task completion without claiming hiring impact unless a suitable longitudinal design supports it.

7.5

Fairness and Accountability Plan

Before any use in screening or career allocation, perform a documented impact assessment. Identify populations potentially affected, decide which dimensions can be lawfully and ethically evaluated, and involve domain experts and user representatives.

Audit input coverage: compare job description completeness and skill extraction quality across sectors and role types. Audit ranking outcomes across relevant profile groups only when data collection is justified, governed, and sufficiently powered.

Provide contestability: users should be able to correct profile details, flag stale or irrelevant jobs, and obtain an explanation of the textual evidence used. Corrections should not automatically become training labels without consent and quality control.

Maintain a model card or system card recording intended use, out-of-scope use, data provenance, known limitations, evaluation populations, and update history. No such completed card is included in the supplied repository.

7.6

Operational Monitoring Plan

If deployed, monitor data freshness, missing-field rates, ranking latency, encoder failures, duplicate postings, and user-reported issues. Monitoring should avoid retaining unnecessary profile content. Changes to the encoder, alias dictionary, job source, or ranking weights should trigger regression evaluation.

Create an incident process for harmful or misleading recommendations. Preserve enough metadata to investigate a reported result while honoring retention limits and privacy obligations. Clearly communicate when a vacancy source or model is unavailable.

7.7

Research Contribution and Limitations

The project contributes a modular prototype for comparing lexical, semantic, and skill-based ranking signals. It also includes skill-gap analysis, O*NET-oriented graph preparation, evaluation tools, and exploratory

metric outputs. The available results do not confirm that the approach improves on its baselines, and they do not establish a validated recommendation product.

This distinction is academically useful: a well-scoped, honest negative or inconclusive result can motivate better experiments. The next research contribution should be built around controlled comparisons and independent labels rather than broader claims unsupported by the current evidence.

Appendix C – Job and Profile Data Dictionary

Field	Meaning	Validation and caveat
job_id	Stable identifier for a vacancy	Required and unique within a snapshot
title	Job title text	Required for hybrid ranker; preserve original wording
job_description	Main vacancy narrative	May be absent or truncated; source and timestamp needed
responsibilities	Role responsibilities text	Optional; avoid double-counting if repeated elsewhere
minimum_qualifications	Required qualifications text	Optional and source-specific
preferred_qualifications	Preferred qualifications text	Keep distinct from mandatory requirements
experience_level	Experience/seniority indicator	Normalize only with an explicit mapping
industry	Industry/category field	Taxonomy and provenance need documentation
normalized_skills	Canonical skill list	Retain source phrase and normalization version
profile_text	Seeker narrative/query	Sensitive personal data; minimize and protect
profile_skills	Structured profile skill list	Presence is not proof of proficiency
label_type	Provenance of relevance judgment	Must distinguish AI suggestion, human rating, adjudication
relevance	Graded or binary job relevance	Rubric and annotator metadata required

Appendix D – Experiment Record Structure

- Experiment ID and objective
- Source data identifier, license/permission status, snapshot timestamp, and file hashes
- Preprocessing code revision and configuration
- Candidate-generation algorithm and fixed candidate IDs
- Profile inclusion rules and profile count
- Relevance rubric, label provenance, number of annotators, agreement, and adjudication
- Model class, model revision/checksum, component weights, and text fields
- Software environment and hardware description
- Random seeds and any nondeterministic operations
- Per-profile outputs, aggregate metrics, uncertainty intervals, and excluded records

- Known deviations from the preregistered or planned protocol

Appendix E – Test and Verification Strategy

Test layer	Example checks	Scope in supplied evidence
Unit	Skill parser handles list strings, aliases, empty and malformed values	Test files present; not run for this report
Metric	Precision/recall/NDCG/MRR boundary cases and zero-relevant profiles	Test file present; no execution claimed
Data contract	Missing columns, duplicate IDs, null title and empty corpus	Validation code exists in modules; full data unavailable
Model behavior	Stable ordering, correct top-k, component scores, empty profile errors	Some checks encoded in source/tests; not independently run here
Graph integrity	All edges reference nodes; IDs unique; mapping confidence retained	Graph builder checks exist; underlying graph files are not in the repository. Saved evaluation mappings are present.
Integration	From source inputs to saved recommendations and report	Streamlit and React/FastAPI source paths are present; full runtime cannot be checked without the processed jobs CSV.
Performance	Latency, memory, batch sizing on representative corpus	No benchmark results verified

Appendix F – Risk Register

Risk	Potential impact	Mitigation direction
Unclear dataset rights	Unauthorized use or redistribution	Verify source and terms; restrict or remove data if needed
Noisy relevance labels	Misleading model comparison	Independent multi-annotator judgment and adjudication
Incomplete candidate pool	Recall estimates biased by selection	Freeze and audit candidate construction; pool across systems
Taxonomy mapping errors	Misleading occupation/skill pathways	Audit mapping precision and flag uncertain links
Skill absence misread as inability	Unfair or discouraging advice	Explain profile evidence limits; enable corrections
Stale postings	Wasted user effort	Track source timestamps and verify vacancy status
Profile privacy exposure	Personal or employment history leakage	Data minimization, access control, retention limits
End-to-end runtime not checked	The processed job dataset is absent, so recommendation and market-insight flows could not be	Restore the data and test both interface paths, including empty/malformed-data cases.

	run from this repository.	
Small profile sample	Unstable aggregate results	Expand sample and report profile-level uncertainty
Model score interpreted as probability	Overconfidence in suitability	Use ranking language; calibrate only if specifically validated

Appendix G – Reproducibility Checklist

- [ ] Exact source dataset and permissions confirmed
- [ ] Dataset schema and source period documented
- [ ] Raw and processed file hashes recorded
- [ ] Candidate sets frozen and versioned
- [ ] Relevance rubric finalized before evaluation
- [ ] Human/AI-assisted labels clearly distinguished
- [ ] Annotator agreement measured where human labels exist
- [ ] Model revision and configuration recorded
- [ ] Input text fields and preprocessing frozen
- [ ] Random seeds and software versions recorded
- [ ] Per-profile metrics exported
- [ ] Confidence intervals or uncertainty limitations stated
- [ ] Graph inputs, mapping coverage, and graph version archived
- [ ] End-to-end application path verified
- [ ] User-facing score explanations checked for overclaiming

Appendix H – Figures and Screenshots from the Supplied Capstone Report

The following dashboard and architecture images are retained from the original capstone report as visual documentation of the project presentation. They complement the source-code description; the technical implementation is described in Chapters 4 and 5.

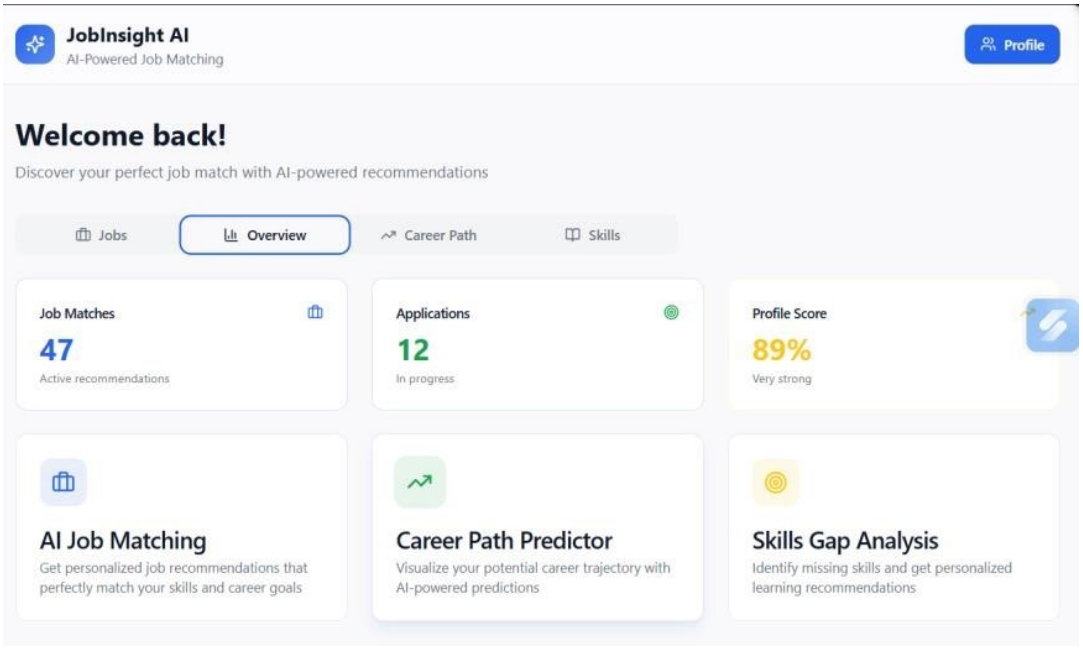


Figure H.1 Dashboard overview shown in the supplied prior report (reference image)

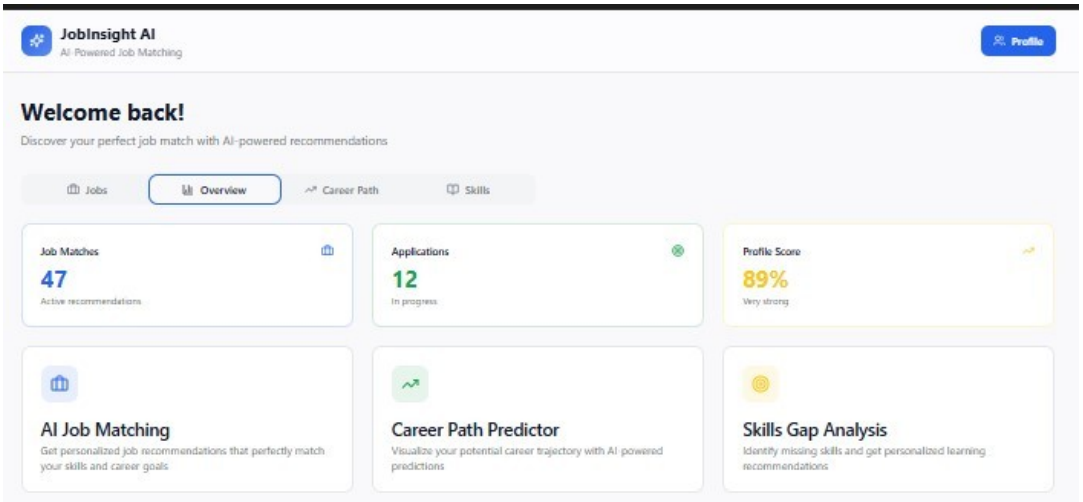


Figure H.2 Job listing interface shown in the supplied prior report (reference image)

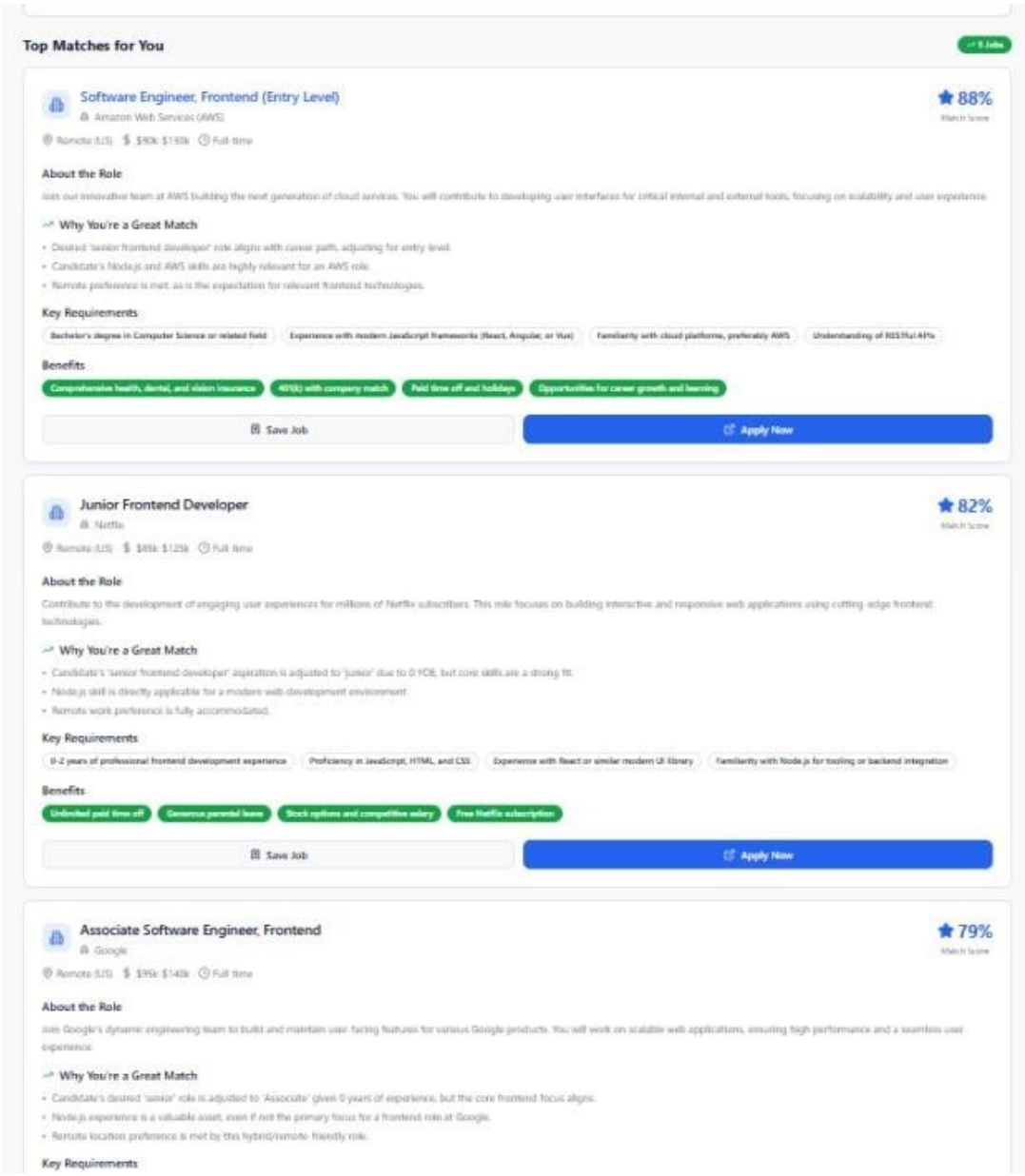


Figure H.3 Skill-analysis interface shown in the supplied prior report (reference image)

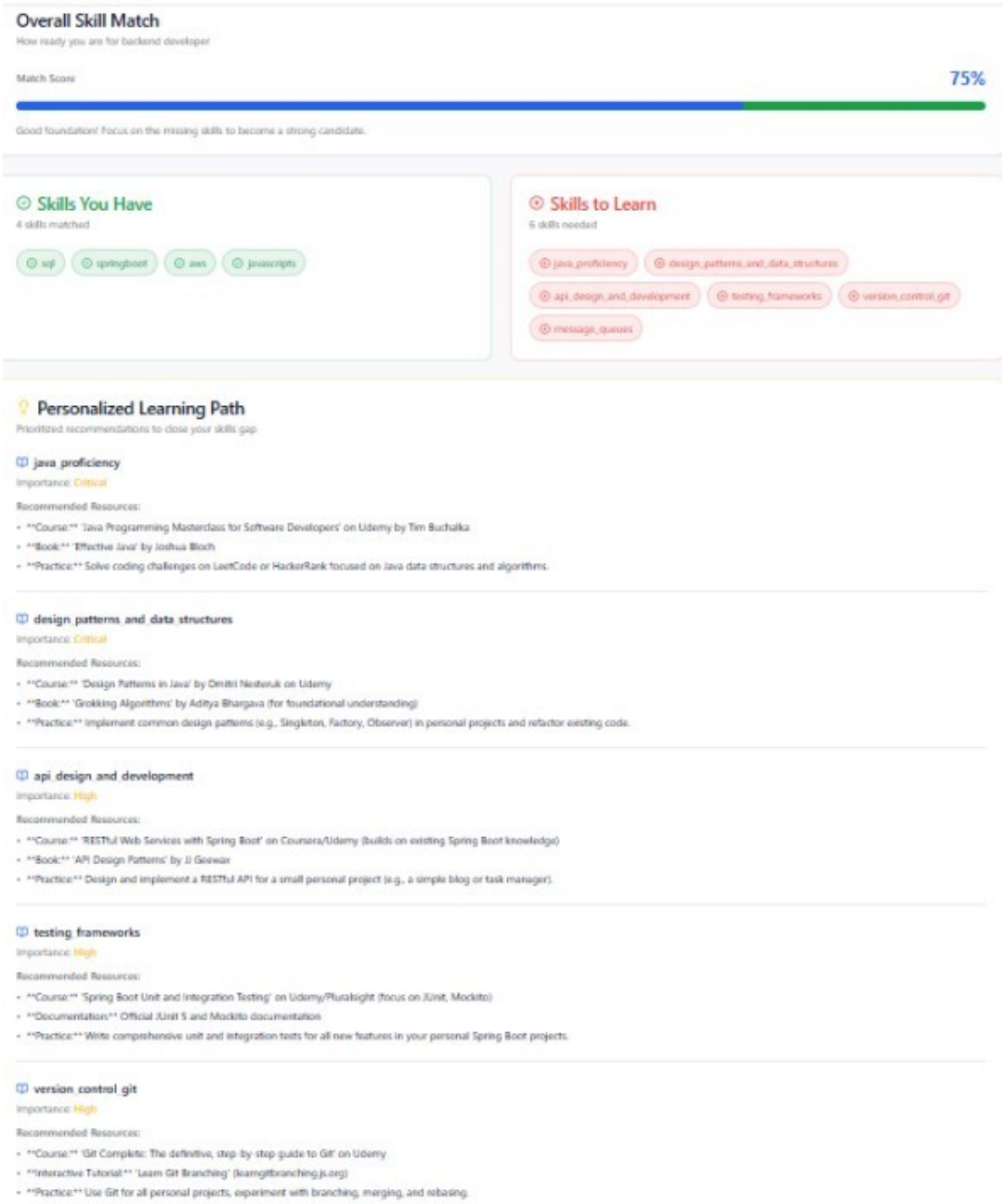


Figure H.4 Profile interface shown in the supplied prior report (reference image)



Figure H.5 Market-analysis interface shown in the supplied prior report (reference image)

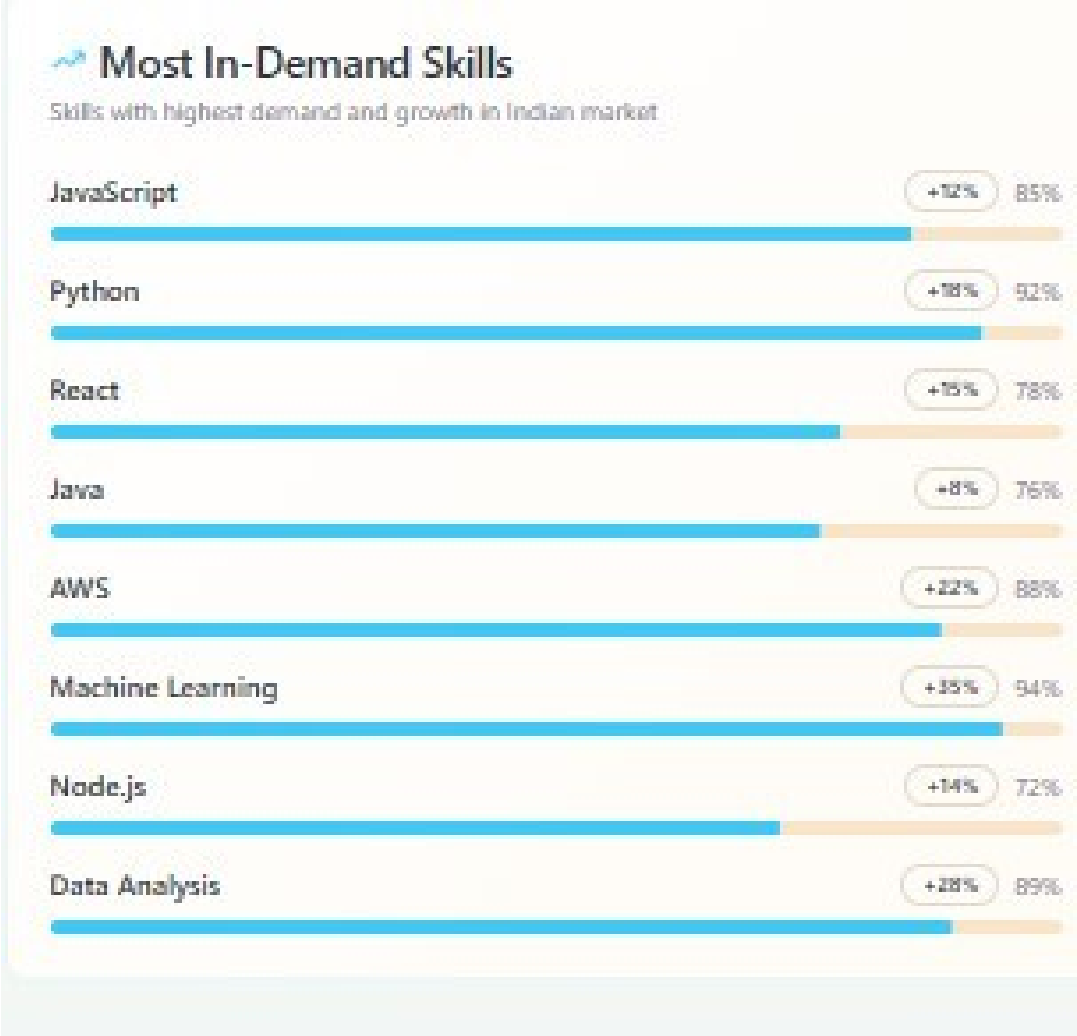


Figure H.6 Skill-demand visualization shown in the supplied prior report (reference image)

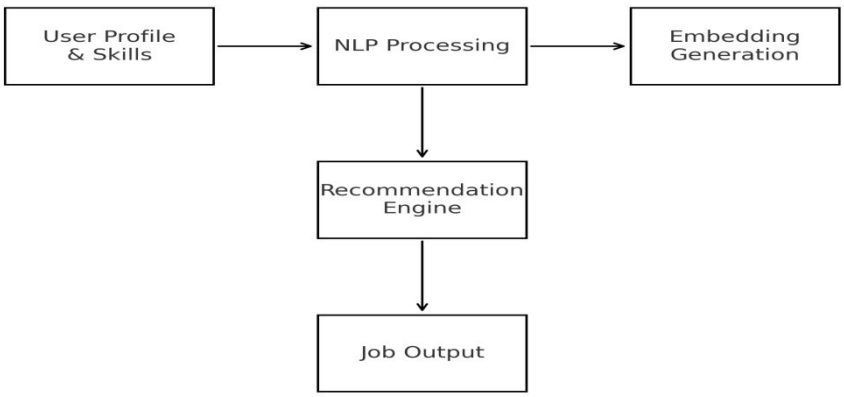


Figure H.7 Earlier workflow diagram from the supplied prior report (reference image)

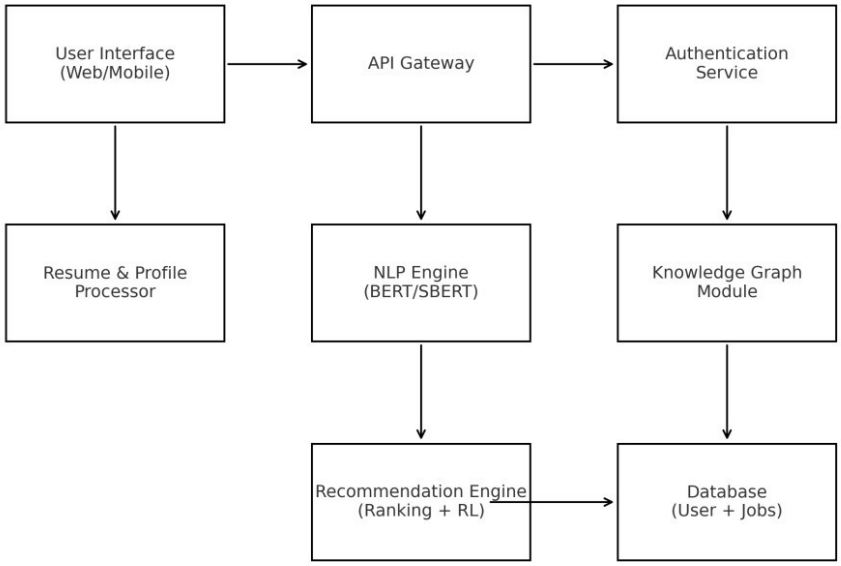


Figure H.8 Earlier architecture diagram from the supplied prior report (reference image)

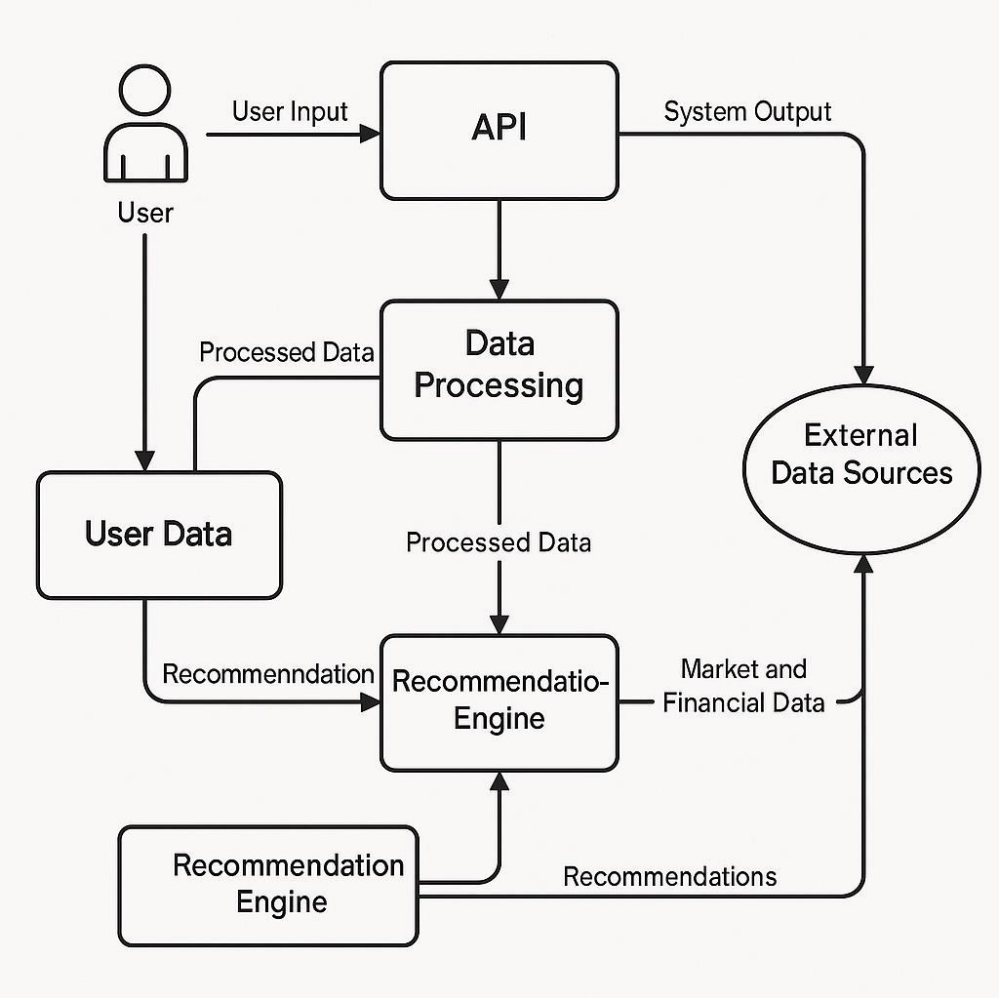


Figure H.9 System architecture diagram retained from the original capstone report

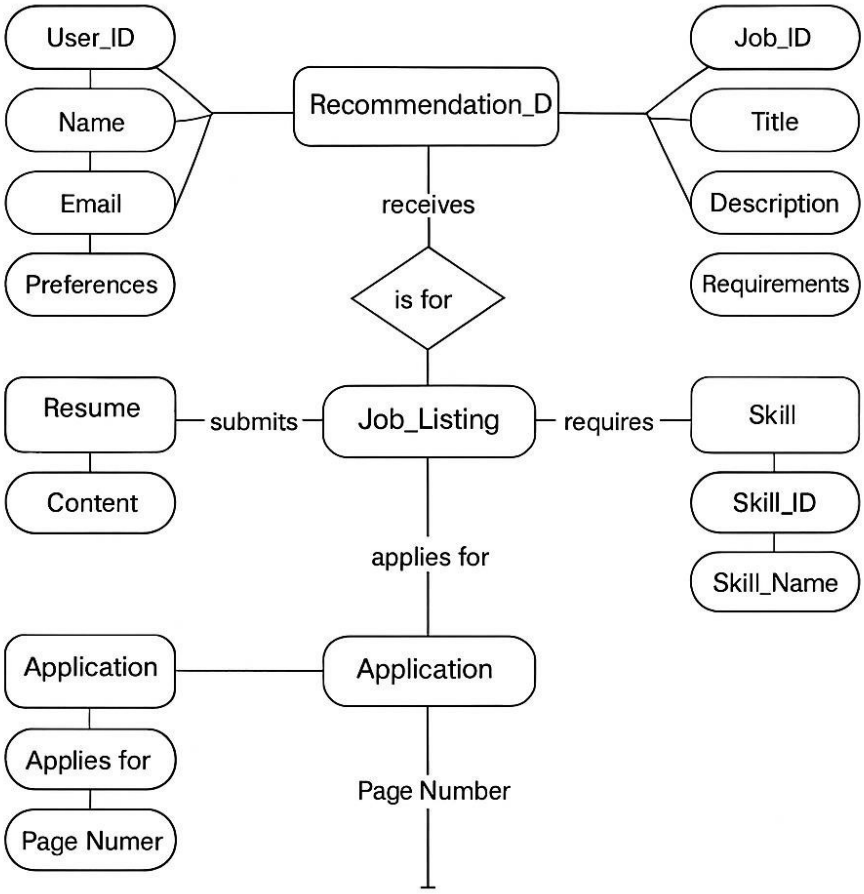


Figure H.10 Entity-relationship diagram retained from the original capstone report

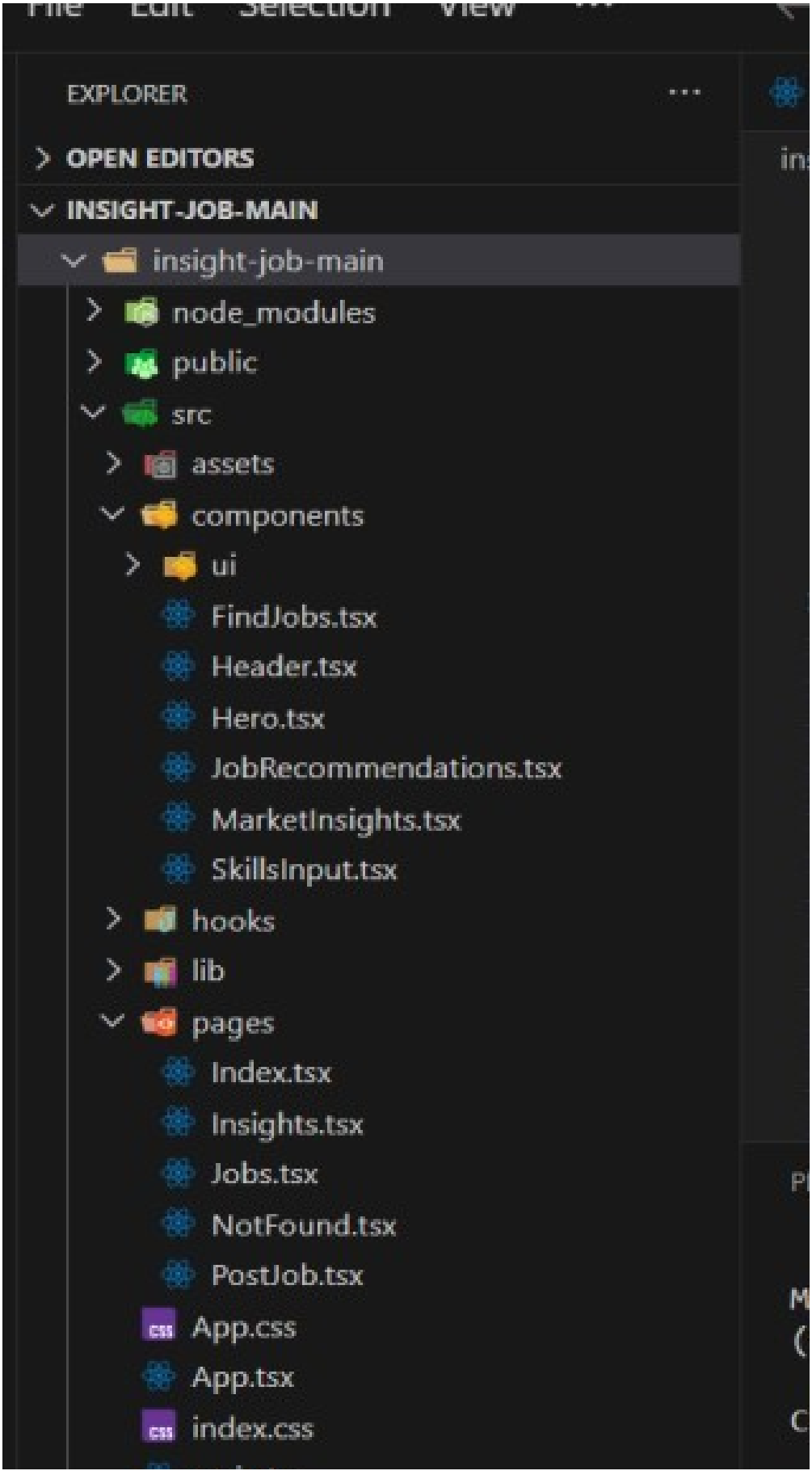

File Edit Selection View
EXPLORER
OPEN EDITORS
INSIGHT-JOB-MAIN
insight-job-main
node_modules
public
src
assets
components
ui
FindJobs.tsx
Header.tsx
Hero.tsx
JobRecommendations.tsx
MarketInsights.tsx
SkillsInput.tsx
hooks
lib
pages
Index.tsx
Insights.tsx
Jobs.tsx
NotFound.tsx
PostJob.tsx
App.css
App.tsx
index.css

Figure H.11 Source-tree screenshot retained from the original capstone report

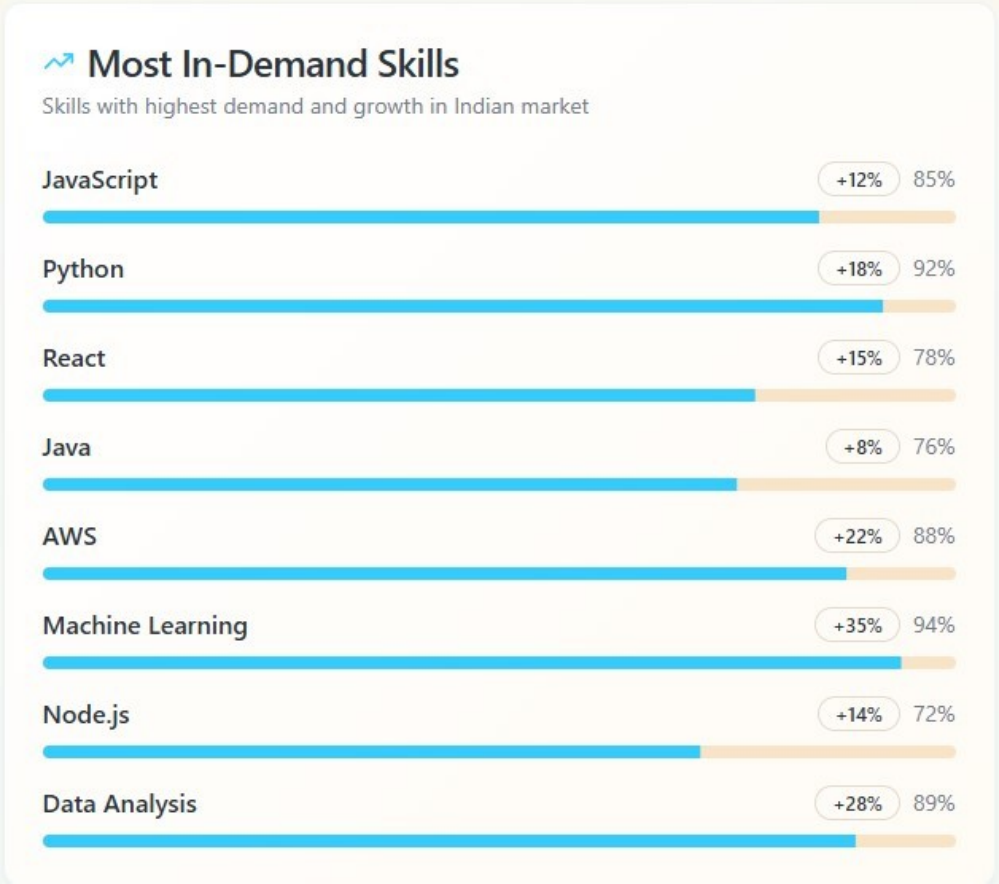


Figure H.12 Skill-demand chart shown in the original capstone report; underlying data are not included in the supplied archive

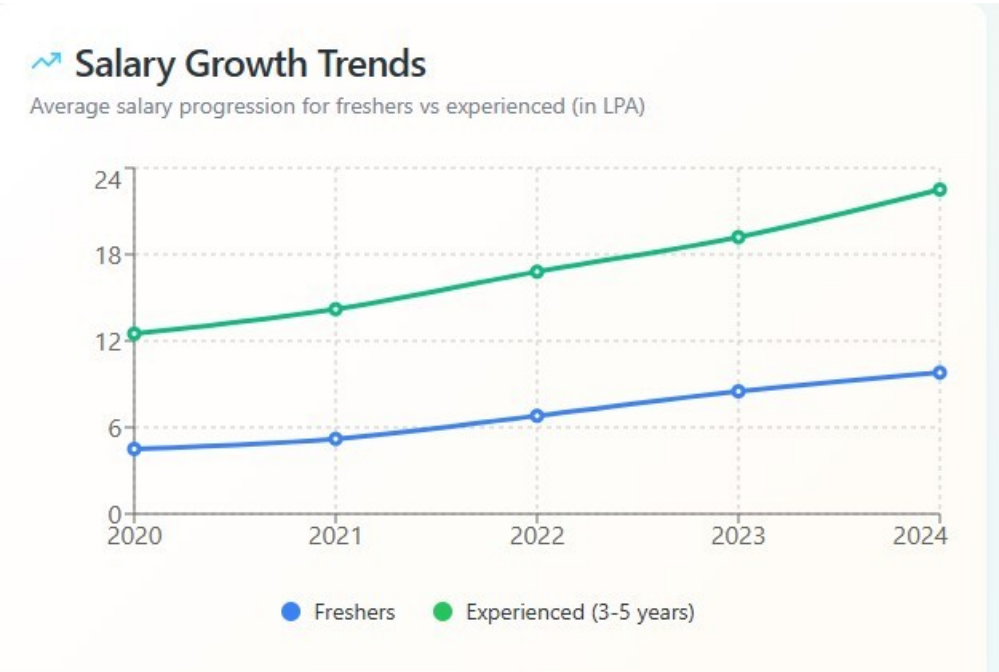


Figure H.13 Salary-trend chart shown in the original capstone report; underlying data are not included in the supplied archive

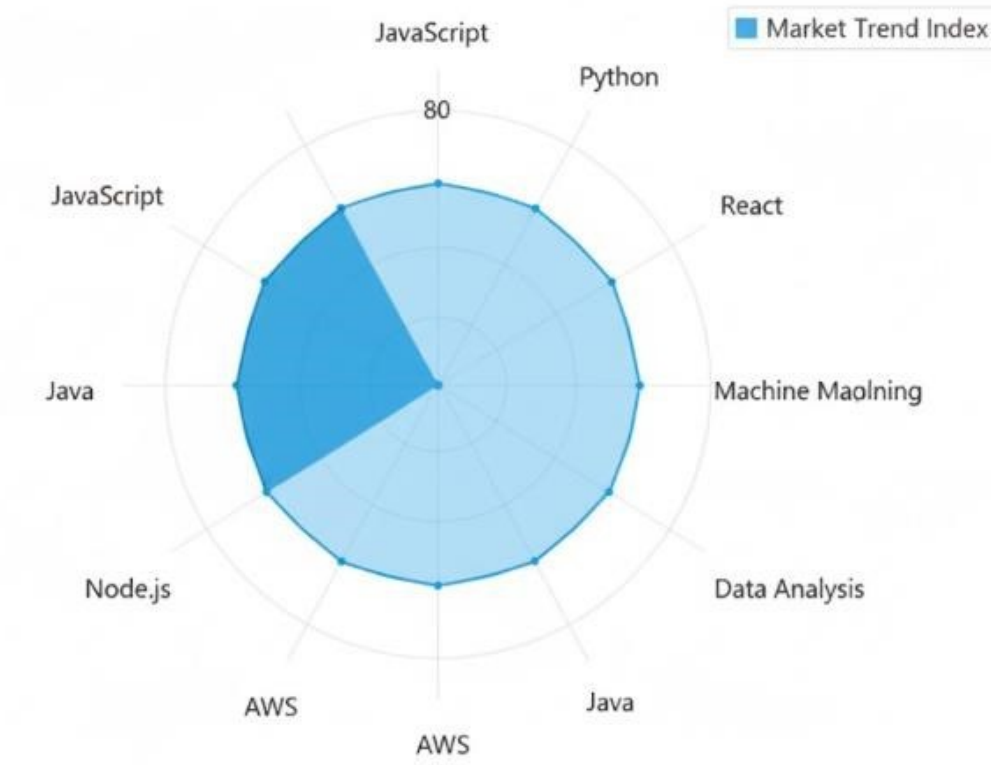


Figure H.14 Skill-profile visualization retained from the original capstone report

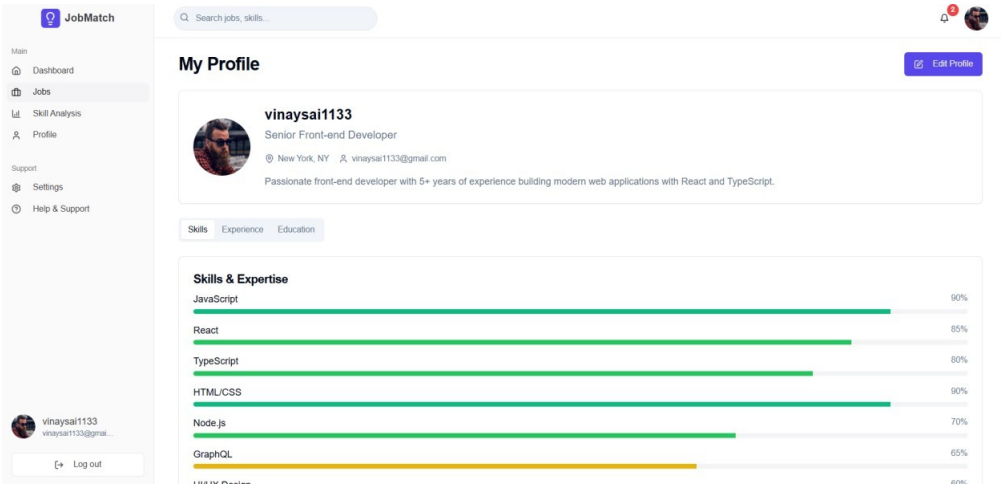


Figure H.15 Profile interface screenshot retained from the original capstone report

Appendix I – Glossary

Term	Meaning in this report
Candidate set	Jobs eligible for ranking and evaluation for one profile
Cosine similarity	Normalized dot-product similarity between vectors

Embedding	Dense numerical representation of a text input
Ground truth	Reference relevance judgments treated as evaluation target; here labels are provisional/AI-assisted
Knowledge graph	Entities linked by typed relations
NDCG	Normalized discounted cumulative gain
Profile	Text and structured skills representing a job seeker query
Ranker	Procedure that orders candidate jobs by a score
Skill gap	Required skill string not present in the parsed profile skill set
TF-IDF	Term frequency–inverse document frequency weighting

Appendix J – Evaluation Protocol

J.1 Purpose and Scope

This protocol is a proposed template for a future controlled study of job ranking. It is included to make the next experiment operationally clear. It does not describe a completed human-subject study. The current repository results remain preliminary, small-sample artifacts with AI-assisted label provenance.

J.2 Benchmark Unit

The unit of analysis should be a seeker profile paired with a fixed candidate set. Each profile must have a stable anonymized identifier, profile text, structured skills, and any declared constraints used by the model. The candidate set must contain stable job IDs, source text fields, and vacancy freshness metadata. A profile may be evaluated only when the candidate and label data are complete enough to apply the stated protocol.

J.3 Inclusion and Exclusion Rules

- Include only jobs for which the source text and posting status are available at the evaluation snapshot.
- Exclude exact duplicate records according to a prespecified identifier or content rule and retain an audit trail.
- Exclude profiles with no usable query evidence only if this rule is stated before model execution.
- Do not exclude difficult or low-scoring examples after seeing model outputs.
- Record every exclusion and its reason at profile and candidate levels.

J.4 Annotation Procedure

Annotators should receive randomized job order, the same profile summary, and the same rubric. Model name, rank, and score should be hidden during relevance judgment. Each candidate should receive a graded rating and optionally a “cannot judge” state. Annotators should record short rationale categories such as role family, core skill overlap, seniority, or missing context; free-text rationale can aid adjudication but may itself contain personal information.

At least two annotators should label an initial calibration subset. Agreement should be measured using a statistic appropriate to the ordinal or categorical scale and interpreted with its assumptions. Disagreement patterns should be discussed and the rubric clarified before labeling the remaining set. A separate adjudicator can resolve differences. The final data should preserve original ratings, adjudicated rating, and provenance.

J.5 Model Comparison Controls

All systems should receive the same profile information and candidate text. If a model uses structured skills while a baseline does not, report both a controlled comparison with common information and a feature-availability comparison. Model weights must be chosen on development data only. The final test set should be evaluated once after the protocol and code are frozen.

J.6 Reporting Template

Item	Report entry
Dataset snapshot and permissions	[Complete with verified source and terms]

Profiles / candidates	[Counts by inclusion rules]
Label scale and annotators	[Rubric, annotator roles, agreement]
Candidate construction	[Retrieval/pooling method and fixed IDs]
Models and configuration	[Code and model revisions; weights]
Metrics	[Per-profile P/R/NDCG/MRR and aggregate]
Uncertainty	[Intervals or explicit reason not estimable]
Limitations	[Label, sample, domain and missingness caveats]

Appendix K – Metric Worked Examples

K.1 Precision and Recall

Suppose a hypothetical top-five list has relevance labels [1, 0, 1, 0, 0] and the candidate set contains four known relevant jobs in total. Precision@5 is 2/5=0.40. Recall@5 is 2/4=0.50. This example only explains arithmetic; it is not a project result.

K.2 Reciprocal Rank

If the first relevant item appears at rank 3, the reciprocal rank for that profile is 1/3. If it appears at rank 1, reciprocal rank is 1. If no relevant item appears, reciprocal rank is 0 under the common convention used by many ranking evaluations. An aggregate MRR is the average over profiles, including zero-hit profiles according to the chosen convention.

K.3 Discounted Gain

For graded labels, DCG gives more credit to higher-ranked relevant items and discounts gain at lower positions. The evaluation script in the archive uses gain 2^rel−1 and a logarithmic rank discount. NDCG compares actual discounted gain to the ideal ordering over the judged candidate set. If candidate labels are incomplete, the ideal denominator and the measured score are also affected.

K.4 Metric Selection

Precision@10 answers whether the first ten are predominantly relevant under labels. Recall@10 emphasizes how many known relevant candidates are surfaced. NDCG@10 captures graded relevance placement when labels have meaningful levels. MRR emphasizes the earliest relevant result. None addresses freshness, user constraints, fairness, explanation fidelity, or the likelihood of interview or hiring outcomes.

Appendix L – Component-Level Implementation Notes

L.1

TF-IDF Path

The baseline code imports scikit-learn’s TfidfVectorizer and cosine similarity utilities. This indicates a sparse lexical vector path. The package-level baseline class accepts an n-gram range and minimum document frequency, fits an identifier/text corpus, and ranks a query against the fitted representation. Exact parameter

values for a specific saved result should be confirmed from its execution configuration; the base configuration sets TF-IDF ngram_range [1,2] and min_df 2.

When minimum document frequency exceeds corpus frequency for a rare vocabulary, terms may be omitted. An n-gram range can capture short phrases but increases feature dimensionality. A controlled evaluation should record fitted corpus and vocabulary statistics and avoid fitting on held-out text if the intended protocol requires a train/test separation.

L.2

Sentence-Transformer Path

The semantic ranker checks that identifiers and texts have matching lengths, IDs are unique, and texts are nonempty. It encodes job documents with normalized embeddings and encodes a query for ranking. Normalization allows cosine similarity through a dot product. Stable sort behavior is used for reproducible tie ordering in the package class.

An operational run should record whether model weights were downloaded or cached, the exact library/model revision, device type, batch size, and encoding parameters. The archive does not include a model artifact or run log that establishes those details for the saved tables.

L.3

Hybrid Path

The hybrid class validates that weights are nonnegative and sum to one. It combines three high-level signals: semantic similarity, profile-to-job skill overlap, and title relevance. Job skills may be collected from fields named normalized_skills, parsed_skills, and skills_required. The parser supports common list encodings and delimited strings and normalizes several aliases.

For jobs without required skills, the implementation must decide how to score the skill component. The source handles empty skill sets as a special condition. Any interpretation should be checked against the exact continuation of the rank function and the evaluation version used to generate a result. This report does not claim that the default configuration was used for every CSV output.

L.4

Skill-Gap Path

The standalone skill-gap analyzer parses job requirements and profile skills, normalizes terms, and calculates profile-to-role differences. Saved CSV and JSON reports accompany this module. Skill-gap values describe text coverage only: a skill absent from the submitted profile is not proof that the user lacks that capability.

When translated to user language, a gap should be phrased as “not found in the supplied profile” rather than “you do not have this skill.” A user may possess the competency but omit it from the profile. A learning recommendation should only follow after verifying that the skill is relevant and the user wants a learning path.

L.5

Graph Mapping Path

The mapping scripts include fuzzy title matching and semantic matching against occupational titles. Such matching can generate candidate occupation links. The build process then connects occupations to skill records and related occupations. A reproducible mapping should retain match method, score, threshold, source occupation ID, and reviewer decision.

Title matching is sensitive to abbreviations, seniority suffixes, geography, and employer-specific titles. Semantic mapping can return plausible but wrong occupation neighbors. Human review of a stratified sample is needed to estimate precision and to select a confidence threshold. No measured mapping quality is contained in the archive.

Appendix M – Requirements Traceability Matrix

Objective	Related requirement	Repository evidence	Evidence gap / scope
Compare lexical and semantic ranking	FR-2, FR-3	TF-IDF and SentenceTransformer modules; saved metrics	Inputs absent; labels preliminary
Combine complementary signals	FR-4	HybridRanker with weighted components	Direct hybrid and graph-hybrid modules exist; saved graph comparison changes multiple weights.
Explain profile-to-role gaps	FR-5	Skill-gap analyzer and report artifacts	Underlying data absent; proficiency not assessed
Represent occupations and skills	FR-6	O*NET mapping and graph-building scripts	Mapping and comparison outputs are saved; source tables and graph build inputs are absent.
Evaluate top-k ranking	FR-7	Metric functions, evaluation scripts and CSVs	Three profiles; provisional/AI-assisted labels; 135-candidate graph comparison.
Provide usable end-user platform	FR-8	Streamlit app.py plus React/Vite frontend and FastAPI backend; SQLite profile API.	Job data missing; full runtime and deployment not verified.
Support labor-market analytics	Future requirement	EDA and graph modules present	No validated time-series or market findings verified

Appendix N – Detailed Risk Analysis

N.1

Data and Label Risk

The first major risk is a mismatch between the claims made and the completeness of the evidence. The repository has metric outputs but not the raw inputs or a robust benchmark manifest. A number can appear precise to four decimals and still have substantial uncertainty due to label noise, candidate selection, and sample size.

The second risk is evaluator dependence. If an AI assistant generated labels using text similarity cues, those labels may align with lexical or semantic rankers for reasons unrelated to human usefulness. Mitigation requires independent annotation, blinded judgments, and agreement reporting.

N.2

Representation and Taxonomy Risk

Skill aliases may be incomplete or ambiguous. A short token may have multiple meanings, and a broad phrase may refer to very different levels of competence. Normalization should keep provenance and confidence, and a taxonomy should be updated with review rather than silently rewriting user language.

Occupation mappings can hide uncertainty if a single best match is stored without alternatives. Retaining multiple candidates and confidence supports review, but downstream ranking needs a predeclared rule to avoid arbitrary propagation.

N.3

User Impact Risk

A recommendation interface may shape a person’s search and self-perception. If the system consistently surfaces a narrow set of roles, it may reduce exploration. A skill-gap list can be interpreted as a judgment of ability. Clear explanations, user controls, diverse retrieval, and a reminder that ranking is exploratory can reduce overreliance.

If an employer repurposes the system to rank applicants, the consequences and legal obligations differ substantially from job-seeker assistance. Such use should be explicitly out of scope until independently assessed and approved.

Appendix O – Architecture Figures for the Completed Research Prototype

The original report’s diagrams are retained as reference images in Appendix H. For future revisions, the following native figures can replace them after the pipeline is implemented and verified:

- Figure O.1 Data lineage from source vacancy to normalized job record
- Figure O.2 Profile and job text construction for each ranker
- Figure O.3 Score fusion and per-component explanation

- Figure O.4 O*NET mapping confidence review workflow
- Figure O.5 Evaluation candidate generation and blinded annotation
- Figure O.6 Reproducibility manifest and artifact relationships

These are proposed figure captions only and are not presented as diagrams of a currently deployed application.

Appendix P – Project Reproducibility Checklist

- [ ] Confirm student names and register IDs against university records
- [ ] Confirm supervisor title and department wording
- [ ] Add official department-approved declaration and certificate language if required
- [ ] Confirm whether the university requires a signed plagiarism/originality statement
- [ ] Update page references and fields in the final DOCX using the university's preferred Word version
- [ ] Confirm all screenshots included are approved for submission and correctly described
- [ ] Obtain and format verified scholarly references actually consulted
- [ ] Add data source, release, and licensing citations only after verification
- [ ] Confirm report length, margins, font, binding margin, and pagination rules with department
- [ ] Review every table, figure, heading, and appendix in print layout
- [ ] Confirm that conclusions match the available independent evidence

Appendix Q – Study Limitations

A concise, evidence-aligned summary suitable for future manuscript use is: "The repository implements lexical, semantic, and weighted hybrid job ranking components and includes skill-gap and occupational graph preparation scripts. Preliminary outputs on three profiles use provisional or AI-assisted relevance labels and do not establish general model superiority. Underlying datasets and graph artifacts were unavailable for independent audit; further evaluation with documented data and independent judgments is required."

Appendix R – Data Governance and Datasheet

R.1 Dataset Description

A dataset datasheet should be completed separately for each source. It should identify the dataset name, publisher or collector, access method, release or snapshot date, geographic and occupational coverage, intended purpose, licensing terms, and any restrictions. The CareerIntel archive includes a DATASHEET_TEMPLATE.md but the template is not completed and data files are not included. This means provenance and licensing remain unknown in the materials reviewed.

R.2 Composition and Collection

The datasheet should document the number of records at each processing stage, fields collected, time range, deduplication procedure, inclusion criteria, and missingness. It should state whether records are real job postings, synthetic examples, manually authored profiles, or a mixture. It should also disclose whether the records were collected from public websites, provided by an organization, or obtained from a licensed source. None of these attributes can be verified from the supplied archive.

R.3 Processing and Labeling

Processing documentation should describe cleaning, skill parsing, alias normalization, occupation matching, and all filtering. Label documentation should identify labelers, rubric, AI assistance, blind status, adjudication, and agreement. In the available result files, aggregate label_type values are visible; the detailed provenance of every judgment and annotator is not independently established here.

R.4 Ethical and Legal Considerations

Job descriptions may contain employer names, contact information, or other sensitive details. Profiles can contain personal histories and potentially sensitive attributes. A governance review should confirm data rights, access controls, retention, secure deletion, and safeguards against unintended use. The report cannot infer permissions from a file’s presence in a repository.

R.5 Known Limitations

The project archive has no actual dataset in data/raw or data/processed. The only verified counts in this report are the profile and candidate counts shown in saved evaluation summaries. Those are evaluation artifact counts, not a confirmed count of the underlying job corpus. The datasheet should not be completed by guessing values from file names or result rows.

Appendix S – Job and Profile Field Handling Matrix

Data element	Potential use	Handling consideration	Evidence scope
Title	Lexical, semantic and title relevance	Preserve exact text; normalize cautiously	Confirmed in ranker source
Description	Main semantic and lexical content	Length, boilerplate and truncation affect representation	Expected field in source code
Responsibilities	Role-specific context	May repeat description; audit	Listed in hybrid text

		duplication	construction
Minimum qualifications	Required capability context	Distinguish mandatory from preferred text	Listed in hybrid text construction
Preferred qualifications	Additional match context	Avoid treating preference as hard constraint	Listed in hybrid text construction
Experience level	Seniority context	Normalize with transparent mapping	Listed in hybrid text construction
Industry	Domain context	Taxonomy and source may vary	Listed in hybrid text construction
Required skills	Explicit skill coverage	Preserve original phrase and canonical skill	Several skill fields parsed
Profile summary	Semantic query representation	Sensitive, user-controlled text	Input to ranker API
Profile skills	Overlap and gap analysis	Absence does not mean inability	Input to hybrid/skill analysis
Location/preferences	Potential filtering/ranking constraints	No verified integrated use	Not confirmed in app
Posting date/status	Freshness and validity	Must be sourced and monitored	Not confirmed in outputs

Appendix T – CareerIntel System Profile

T.1 Model Identity

System: CareerIntel experimental ranking components. Model families: TF-IDF baseline, SentenceTransformer semantic ranker, and weighted hybrid ranker. The hybrid class default encoder identifier is sentence-transformers/all-MiniLM-L6-v2. A revision checksum is not present in the supplied archive.

T.2 Intended Use

Exploratory retrieval of candidate job postings for research and job-seeker decision support. Ranking scores are not employment probabilities, qualification determinations, or employer selection recommendations.

T.3 Out-of-Scope Use

Automated applicant rejection, hiring decisions, claims about likely employment success, demographic inference, or labor-market forecasting are not supported by the evidence available for this report.

T.4 Data and Evaluation

The processed job and O*NET inputs are absent from the GitHub repository. Saved comparisons cover three profiles with candidate counts of 178, 180, and 135 in different experiment files; labels are provisional_assistant or preliminary_ai_assisted_review. The repository does not establish independent human validation.

T.5 Limitations

The small profile count, uncertain candidate construction, AI-assisted labels, absent raw input data, and unverified graph integration constrain all quality claims. No latency, fairness, calibration, user satisfaction, or production safety results are available.

T.6 Change Management

Future changes to model revision, text fields, skill aliases, score weights, candidate sources, or graph mapping should be versioned and evaluated against a frozen benchmark. These controls define the evidence needed for future revisions.

Appendix U – Interface Content and Explanation Guidance

U.1

Recommended User-Facing Labels

Use “text match” or “ranking score” to describe similarity. Avoid “probability of getting hired,” “best candidate,” or “guaranteed fit.”

Use “skills found in your profile” and “skills listed in this posting but not found in your profile.” This wording communicates the comparison without making a judgment about actual ability.

Show the job source and publication or verification date when available. If freshness is unknown, state that status rather than presenting the posting as current.

Explain the basis for the ranking using actual model components. If only semantic similarity is available, do not imply graph reasoning or employer endorsement.

U.2 User Controls

A future interface should let users edit profile text, remove or correct parsed skills, set explicit preferences, and report an irrelevant or stale listing. A user should be able to understand whether their input is stored and how to remove it. These controls are design recommendations; the archive does not verify a deployed interface implementing them.

U.3 Accessibility and Inclusion

A deployed interface should use keyboard-accessible controls, adequate contrast, readable labels, screen-reader descriptions for charts, and plain-language explanations. It should not rely solely on color to communicate match categories. No accessibility audit or user study is included in the supplied project artifacts.

Appendix V – Research Roadmap and Deliverables

Work package	Concrete deliverable	Exit criterion
WP1 Data recovery	Completed datasheet, source manifests, schema and permissions	Inputs and terms independently verified

WP2 Benchmark	Frozen candidates, rubric, independent labels, agreement report	Labels have documented provenance and adjudication
WP3 Baselines	Reproducible TF-IDF and semantic runs	Same profile/candidate inputs; run manifests complete
WP4 Hybrid ablation	Signal ablations and weight sensitivity results	Conclusions robustly reported with uncertainty
WP5 Graph evaluation	Audited graph and graph/non-graph comparison	Mapping accuracy and coverage measured
WP6 Application	Integrated user-facing prototype and interface tests	End-to-end flow demonstrably operates on verified data
WP7 Responsible pilot	Protocol, consent and usability/fairness analysis	Approved, bounded pilot with clear outcome limits

Appendix W – Source-to-Claim Traceability

Report claim	Primary repository evidence	Qualification
Hybrid combines semantic, skill, title signals	src/careerintel/models/hybrid_ranker.py	The Streamlit and FastAPI recommendation routes call the ranker code; full runtime depends on the missing job data.
Default weights 0.60/0.30/0.10	HybridRanker constructor	Defaults may differ from evaluation script configuration
Semantic encoder configured by model name	base.yaml and semantic ranker source	No model artifact or revision hash included
Graph builder expects occupation and skill files	src/knowledge_graph/build_graph.py	Graph scripts and saved evaluation mappings/metrics are present; O*NET source tables and graph input files are absent.
Three profiles, 178 candidates in saved TF-IDF/semantic/hybrid comparison	Aggregate evaluation CSVs	Stored labels provisional assistant-generated
Three profiles, 180 candidates in preliminary AI-assisted review comparison	Reviewed comparison aggregate CSV	Label type says preliminary AI-assisted review
Interactive application source is present	app.py; frontend/src/App.jsx; backend/main.py and routers	Processed jobs CSV is absent; full runtime and deployment were not verified.
Underlying data provenance is unknown	Raw/processed folders and datasheet template	No actual source dataset or completed datasheet included

Appendix X – Glossary of Evaluation and Data Terms

Term	Definition
Adjudication	Resolution of differences between independent annotator judgments
Ablation	Controlled removal of one model component to estimate its contribution
Candidate pooling	Combining candidates retrieved by multiple systems for evaluation
Cold start	Limited evidence for ranking a new profile or new job
Graded relevance	Ordinal or numeric levels indicating different strengths of relevance
Inter-rater agreement	Consistency among independent annotators before adjudication
Label leakage	Evaluation influence from information or judgments related to model output
Mapping coverage	Share of target entities receiving an accepted taxonomy mapping
Model revision	Specific version of model weights and configuration
Snapshot	Fixed, dated copy of input records used for one experiment

Appendix Y – Expanded Limitations Statement

The main limitations are the missing job and O*NET input data, three-profile evaluation sets, candidate pools whose construction cannot be checked from the source data, and relevance labels marked provisional assistant-generated or preliminary AI-assisted review. Graph-hybrid metrics are saved, but their input graph files are missing and the graph comparison changes more than one score weight. The applications are present in source code, but the missing processed job file prevents a complete end-to-end run. The repository contains no user study, deployment measurements, fairness evaluation, or performance benchmark. I report the saved metrics as artifacts, not as independent or generalisable results.

Appendix Z – Submission Details

The following submission details should be checked by the authors and supervisor before filing:

- Verify whether the final report should include the third student appearing in the old template; the present cover and declaration follow the two students explicitly confirmed in the user request.
- Confirm the official university name, address, degree wording, department naming, supervisor designation, and approved certificate/declaration language.
- Confirm that retaining historical dashboard screenshots is appropriate. They are captioned as reference images and do not establish current application functionality.
- Replace this note with the department’s required final submission checklist or remove it after the authors complete their verification.

Appendix AA – Test Cases

The following cases define validation conditions for the ranking and evaluation pipeline. The project includes unit-test files covering metrics, ranking baselines, skill normalization, and skill-gap analysis.

ID	Input condition	Expected behavior	Purpose
TC-01	Empty candidate list	Clear validation error or empty result with explicit status	Prevent silent failure
TC-02	Profile text is blank	Reject query with actionable message	Prevent meaningless ranking
TC-03	Job ID duplicated	Detect duplicate or document stable disambiguation	Preserve identity
TC-04	Skill field is null	Treat as unavailable/empty without crash	Robust missing data
TC-05	Skill field is Python list string	Parse list without splitting multiword skill	Correct tokenization
TC-06	Skill text uses comma/semicolon delimiters	Parse expected terms and normalize known aliases	Normalization behavior
TC-07	No required skills for a job	Apply documented neutral/fallback skill score	Avoid divide-by-zero or unfair penalty
TC-08	Top-k exceeds candidate count	Return all valid jobs, document metric convention	Boundary handling
TC-09	Two jobs have equal scores	Use deterministic stable tie order	Repeatability
TC-10	Encoder unavailable	Report model-load error without partial silent output	Operational transparency
TC-11	Evaluation label missing	Count and report missing pair; do not silently assume irrelevance	Label integrity
TC-12	Candidate has no matching graph node	Flag unmapped state; do not infer a relation	Graph integrity

Appendix AB – Detailed Change Log Template

Version	Change type	Description	Evidence and approval
0.1	Data	New source snapshot, schema, or permission change	Manifest and rights check
0.2	Preprocessing	Changed parser, alias map, cleaning, or field selection	Unit tests and before/after counts
0.3	Model	Changed encoder, TF-IDF parameters, weights, or title signal	Frozen benchmark comparison
0.4	Graph	Changed occupation data, mappings, or edge definitions	Graph audit and mapping precision sample
0.5	Evaluation	Changed candidate generation, labels, rubric, or	Protocol revision and comparability note

		metrics	
1.0	Application	Changed interface, data flow, or deployment behavior	Integration and user-facing review

Each version should record the change and its effect on comparability. Results from different data snapshots, model configurations, or labeling protocols should remain clearly distinguished.

Appendix AC – Experimental Run Record

Run identifier: _____

Date and time (with timezone): _____

Code revision: _____

Input snapshot identifier and hashes: _____

Model class and revision: _____

Configuration and score weights: _____

Candidate-generation method and candidate-set identifier: _____

Profile count and candidate count: _____

Label provenance and rubric version: _____

Output file paths and hashes: _____

Warnings, excluded records, or deviations: _____

Operator/reviewer: _____

Appendix AD – Discussion Questions for Oral Defense

- Why can a semantic similarity score not be interpreted directly as a probability of job suitability?
- How does a candidate set affect recall and NDCG?
- Why are AI-assisted relevance judgments not independent ground truth for comparing language-based rankers?
- What controls are needed to demonstrate that graph-derived relations add value beyond semantic similarity and skill overlap?
- What does a missing skill in a profile imply, and what does it not imply?
- Why is a comparison over three profiles insufficient to claim broad generalization?
- What data provenance and licensing facts must be established before the dataset can be used or redistributed?
- Which source files demonstrate the hybrid architecture, and what evidence is missing for a deployed application?
- How would you design an ablation that isolates the contribution of the skill signal?
- What safeguards would be needed before adapting a job-seeker recommendation tool to applicant screening?

Appendix AE – Project Completion and Technical Scope

The repository has characteristics of an evolving research project: its README points to planned modules, its research plan lists experiments as pending, its data folders are placeholders, and its result files contain exploratory metric summaries. This combination is common during iterative development, but a formal report should separate planned, implemented, executed, and validated states. In this document, “implemented” refers to source-code behavior visible in the archive; “executed artifact” refers to a saved output; “validated” is reserved for evidence that establishes correctness or generalizable quality. Those categories are not interchangeable.

A saved result may come from an earlier run whose inputs are no longer in the repository. A graph schema does not by itself show that the graph improves recommendations. In this project, both application interfaces and graph-comparison outputs are present, but the missing job and graph inputs limit what can be rerun and verified.

Appendix AF – Technical Summary

The current repository contains TF-IDF and semantic rankers, a direct hybrid ranker, a graph-hybrid scorer, skill-gap analysis, O*NET mapping scripts, evaluation utilities, a Streamlit app, and a React/FastAPI interface with SQLite profile storage. Saved graph-comparison metrics are available, but the input job and graph data are missing, the labels are provisional or AI-assisted, and the compared graph models use different weights. The repository does not establish production deployment, independently validated relevance, real-time market trends, user benefit, or improved employment outcomes.

Appendix AG – Design Trade-Offs

AG.1 Lexical Versus Semantic Representation

Lexical ranking offers transparency at the term level and can be efficient over sparse vectors. Its weakness is vocabulary mismatch. Semantic ranking can associate related phrases but may obscure which terms influenced an embedding score and can inherit domain biases from pretraining. A practical comparison should examine both retrieval quality and explanation needs, and should ensure the systems receive equivalent text evidence.

AG.2 Explicit Skills Versus Free Text

Structured skills make overlap and gaps easier to inspect, but they require reliable extraction and canonicalization. Free text preserves context and nuance but is harder to compare directly. A hybrid design can use both, provided that skill extraction errors are measured and the same competency is not inadvertently counted multiple times through both text and structured channels.

AG.3 Fixed Weights Versus Learned Weights

Fixed weights are simple, reproducible, and suitable for a prototype. Their values encode design assumptions and should be tested for sensitivity. Learned weights may improve ranking when enough representative judgments exist, but they can overfit small samples and require careful train/test separation. Given only three profiles in saved aggregate comparisons, learned-to-rank claims would be premature.

AG.4 Static Taxonomy Versus Dynamic Market Data

An occupational taxonomy provides stable identifiers and curated relations but may lag emerging roles and local terminology. Dynamic vacancy text can reflect current language but is noisy and transient. Combining the two requires release/version tracking and clear handling of stale or unmapped records. No verified time-series labor-market analysis is present in the project archive.

Appendix AH – Decision Log for Evidence-Constrained Reporting

Reporting choice	Reason
Use the original capstone title on cover	Explicitly confirmed project title
Describe CareerIntel as a research extension	Repository README defines that relationship
Do not claim BERT or reinforcement learning in application	No supporting implementation evidence in archive
Do not report a verified dataset size	No job and O*NET input data; saved results exist but cannot be independently reproduced
Show saved metric numbers with label caveats	Aggregate CSVs contain values and label_type
Do not claim a winner	Only three profiles; assistant-generated labels; graph comparison changes more than one weight.
Caption old screenshots as historical references	Images are from the earlier report and do not document the current interface source.
Do not invent scholarly references	Repository has no verified bibliography or external sources

	supplied

Appendix AI – Expanded Research Questions and Operational Definitions

AI.1 Semantic Ranking

Operational comparison: evaluate TF-IDF and semantic scores on the same profile/candidate pairs, with an identical text field policy and held-out labels. Primary outcome: NDCG@10; secondary outcomes: Precision@10, Recall@10, and MRR. Report profile-level outcomes and confidence limitations. Hypothesis direction should be registered before final evaluation, but no conclusion should be forced if results are mixed.

AI.2 Graph Contribution

Operational comparison: hold the semantic and skill baseline constant and add only a graph-derived component. Define graph edge sources, confidence thresholds, path traversal, and score normalization. Evaluate on the same candidate set and separately on a predefined sparse-profile subset. Measure graph mapping quality independently so that a ranking result is not attributed to a noisy graph without audit.

AI.3 Skill-Gap Validation

Operational comparison: mask known profile skills or use independently verified skill evidence, then compare predicted missing-skill lists with the held-out set. A frequency baseline should be specified before comparison. Suitable measures may include precision, recall, or ranking quality over suggested skills, depending on the product question. Masking evaluates recoverability from observed context and does not prove real-world proficiency.

AI.4 Feedback Simulation

Operational comparison: if simulated interactions are studied, document the user-choice model and its assumptions. Evaluate robustness to position bias and random noise. Clearly label results as simulation and avoid implying that a simulated click is an actual user preference. A real feedback study would need a distinct design and consent process.

Appendix AJ – Evidence and Interpretation Framework

Evidence level	Meaning	Example in this report
Direct source evidence	Visible in a supplied source file or saved output	HybridRanker default weights in source
Artifact summary	Metric or count transcribed from saved output	Saved graph-v3 comparison: three profiles, 135 candidates, provisional_assistant labels.
Inferred design description	Reasonable interpretation of modules and dependencies	Conceptual processing flow

Unverified claim	Not demonstrated by source or result files	Production deployment and user-study outcomes
Proposed future requirement	Recommendation for work not shown as complete	Independent labels and graph ablation

This framework separates direct source evidence, saved experimental artifacts, design interpretation, and future research requirements. The distinctions help readers understand the basis and limits of each technical claim.

Appendix AK – Evaluation Quality Criteria

- [ ] Is the candidate set identical for every model in a comparison?
- [ ] Are relevance labels independent of the ranking output being assessed?
- [ ] Are AI-assisted or heuristic judgments explicitly identified?
- [ ] Is the number of evaluated profiles large enough for the intended claim?
- [ ] Are per-profile results available alongside aggregate means?
- [ ] Are missing judgments, excluded pairs, and zero-relevant cases documented?
- [ ] Are metric definitions and gain conventions stated?
- [ ] Are model configuration, model revision, and text fields reproducible?
- [ ] Are confidence intervals or limits on inference explained?
- [ ] Does the conclusion avoid claims about hiring success or user impact without evidence?

If one or more answers are no, present the comparison as exploratory and describe the limitation directly.

Appendix AL – Final Interpretation of the Saved Comparisons

The two aggregate comparisons answer different narrow questions over small, labeled candidate sets. The provisional assistant-labeled outputs show higher NDCG@10 and MRR for the hybrid configuration in that artifact, while semantic and hybrid tie on precision and recall. The preliminary AI-assisted reviewed comparison reports identical aggregate measures for TF-IDF, Semantic, and Hybrid. Together, these outputs do not support a stable claim that a particular model is superior. They do support the conclusion that the evaluation pipeline has produced comparative artifacts and that stronger, independent testing is needed.

The project should preserve these results with their label provenance and avoid silently replacing them with newer values. Future reports should add a run manifest and clearly identify whether a number comes from a saved output, a recomputed run, or an independently validated study.

Appendix AM – Evaluation Protocol Decisions

AM.1 Decision Record

A decision record helps prevent an exploratory comparison from turning into an unsupported performance claim. Before a new ranking experiment, the research team should write down the question, the unit of analysis, the candidate universe, the relevance rubric, the primary metric, the comparison models, and the conditions under which the result will be considered inconclusive.

AM.2 Candidate Universe

The candidate universe should represent the job opportunities that could realistically be shown for a profile at a defined snapshot. It should include a broad retrieval pool rather than only examples already selected by one model. Each candidate should have a stable ID and enough information for annotation. The experiment should report the number of candidates per profile and disclose any limits imposed by missing or stale records.

AM.3 Relevance Judgment

The relevance judgment should be based on an explicit rubric and independent of the model score. Annotators should distinguish role-family fit from exact skill coverage and should have a way to mark insufficient information. When assisted labeling is used, the assistance process should be documented, and a human audit should estimate error before the labels are used for a quality claim.

AM.4 Interpretation Rule

A model difference should be described only when the candidate set, profile inputs, labeling process, and metric calculation are comparable across systems. A numerical lead on one metric can coexist with a tie or reversal on another. The conclusion should describe the observed pattern and uncertainty, not reduce a multi-metric, small-sample comparison to a single winner.

Appendix AN – Model Comparison Reading Guide

When reading a model-comparison table, first inspect the number of profiles, the candidate count, and label provenance. Next compare per-profile results to see whether the aggregate is consistent or driven by one case. Then identify whether all models ranked the same candidates using the same input fields. Finally, interpret metrics according to what they reward: early relevant results, coverage of known relevant items, graded ordering, or first-hit position.

A high NDCG score against a provisional label set means the ordering aligned well with the ideal ordering defined by that label set. It does not establish that users would prefer the ranking, that all suitable jobs were candidates, or that the system improves employment outcomes. A high precision score can coexist with low recall if only a small portion of relevant jobs appear near the top. A high MRR can result from an early first hit while later relevant jobs are poorly ordered.

The CareerIntel artifacts should therefore be read as exploratory diagnostics. The provisional comparison offers a hypothesis for further evaluation, not a conclusion. The identical reviewed-comparison aggregates call for inspecting the item-level ranking and label process before drawing any interpretation. Both observations reinforce the need for frozen data, independent labels, and a sufficiently varied profile sample.

Appendix AO – Interpreting Similarity Components

Lexical similarity rewards shared weighted terms and phrases. A profile and posting that use different but related vocabulary may receive a lower lexical score even when a human sees a plausible relationship. Conversely, a posting that repeats generic terms may obtain lexical overlap without being a strong role match. Vocabulary inspection and query construction therefore matter when interpreting TF-IDF outputs.

Semantic similarity can recognize contextual association, but it can also connect texts through broad topical language that is not sufficient for role relevance. Embeddings compress text into a vector, so the score does not expose a direct list of causal terms. Explanations should use separately computed evidence, such as shared skills or matched phrases, and should not imply that an embedding score itself identifies a specific reason.

Skill overlap provides a discrete and relatively legible signal. Its limitations include incomplete skill extraction, alias gaps, list formatting errors, and a lack of proficiency levels. A required skill list can also be noisy or incomplete. The score should therefore be considered one perspective on profile-job alignment rather than a definitive measure.

Title relevance can help distinguish roles when descriptions are similar, but titles vary across employers and may encode seniority inconsistently. The implementation's title component should be evaluated separately to understand whether it improves ordering or simply reinforces semantic similarity. A title-only ablation and an error review can make this contribution more visible.

The hybrid score offers a compact ordering from these signals. Its weighted average can improve some metrics while reducing others, depending on label quality and candidate mix. Component-level reporting is valuable because it makes a final score easier to inspect and helps identify cases where one signal dominates or conflicts with the others.

Appendix AP – Experimental Reporting Criteria

- State the research question and primary outcome before presenting a table.
- Name the exact input snapshot and candidate-generation process.
- Distinguish all verified human labels from AI-assisted suggestions and heuristics.
- Report the number of profiles and candidate jobs next to every metric.
- Include per-profile results or a distribution plot where the sample supports it.
- Describe any data filtering, missing values, duplicates, and excluded pairs.
- Report component weights and whether they were defaults or tuned values.
- Separate ranking quality from runtime, usability, fairness, and employment outcomes.
- Use uncertainty language when the profile sample is small.
- Keep the conclusion proportionate to the actual experiment.

A well-reported exploratory experiment is useful even when it does not establish a winner. It can show where labels are ambiguous, where candidate generation is weak, and which model components deserve a more controlled test. That role is appropriate for the present saved outputs.

Appendix AQ – Project Contribution and Evidence-Based Conclusions

CareerIntel's completed contribution is a modular research prototype combining lexical, semantic, and weighted hybrid ranking with skill-gap analysis, occupation-mapping scripts, and comparative evaluation tools. The saved experiments document initial ranking behavior on three profiles and identify concrete next steps for independent relevance judgments and graph-signal ablation. Their results are interpreted with the supplied label provenance and candidate-set scope; broader user, hiring, or labour-market conclusions require dedicated studies.